

PROJECT DOCUMENTATION

ScholarLens

Complete Project Manual

From first setup to ingestion, hybrid retrieval, reranking, graph orchestration, API integration, and operations.

Code-reviewed edition

Repository: `rag-papeer-project-main`

Package: `scholarlens` 0.1.0 / API 0.2.0

Review date: 16 September 2026

Evidence boundary

Based on the application source, frontend, deployment configuration, evaluation scripts, and supporting artifacts in the supplied snapshot. Includes verified code behavior, operating recipes, exact references, and unresolved findings. External AI services were not invoked during this review. Runtime evidence and known limitations are identified explicitly.

Contents

1. Scope, evidence, and how to use this manual	1
2. What ScholarLens does	1
3. Repository map and architecture	2
4. Setup from zero: choose a deployment path	3
5. First user workflow and UI guide	5
6. Ingestion and document formats	6
7. Embeddings, Qdrant, and hybrid retrieval	7
8. The reranker: what this version adds	7
9. LangGraph routing and state	8
10. Answer generation, web search, and claim verification	9
11. Provider selection and configuration precedence	10
12. HTTP API and streaming contract	12
13. Session memory, persistence, and recovery	13
14. Docker and deployment	14
15. Frontend internals and operating behavior	15
16. Evaluation, sample data, and smoke checks	16
17. Troubleshooting and operating checklist	17
18. Review findings and prioritized improvements	18
19. Developer maintenance and extension guide	20
A. Exact API endpoint and request reference	21
B. Complete state and structured-output schemas	22
C. Environment-variable reference	24
D. Exact build and deployment configuration	26
E. Complete Python function and class index	31
F. Frontend function and style reference	39
G. Dependency and artifact provenance	41
H. Review coverage, validation, and source fingerprints	46

1. Scope, evidence, and how to use this manual

This manual documents the project supplied in `just-add-reranker-aug-06-10pm-novenv`. The actual application root is two levels below that folder:

```
/home/punam/Music/IMPFolder/Project/
just-add-reranker-aug-06-10pm-novenv/
  just-add-reranker-aug-06-10pm/
    rag-papeer-project-main/    <- application root
    docs/                     <- this documentation
```

Unless stated otherwise, run application commands in `rag-papeer-project-main`, where `pyproject.toml`, `.env`, and `docker-compose.yml` live. Starting in the outer wrapper directory will not find the application configuration.

The review covers all 28 first-party Python files, the complete React JSX and CSS, HTML and favicon, Docker/Nginx configuration, dependency manifests and lockfiles, evaluation scripts and fixtures, README/setup notes, and the differences between the three backend trees. The sample PDF and stored graph image were inspected as supporting artifacts. Runtime database schemas, evaluation-result aggregates, and cache inventory were inspected without reproducing conversation contents or credential values.

Third-party `node_modules`, generated frontend bundles, Python bytecode, Git internals, embedding-cache payloads, and serialized conversation contents are not individually documented as application source. Existing `.env` values are deliberately not copied into this distributable manual. The environment reference is derived from code and Compose, using placeholders for secrets. These boundaries are important: a complete project manual should explain how runtime artifacts work without embedding account credentials or private chats.

Evidence labels:

- **Implemented:** directly established by the reviewed source or configuration.
- **Existing artifact:** a result or file included in the snapshot, without proof that it represents the current code.
- **Review finding:** a code-based limitation or likely failure, not necessarily reproduced against live services.
- **Recommended procedure:** a proposed operating step or improvement, distinct from an existing feature.

The React production build passed using the installed frontend dependencies and an output directory under `/tmp`. All Python files parsed successfully, and eleven offline checks passed using isolated source functions and deterministic stubs. The environment lacks FastAPI, LangGraph, LangChain Core, and PyTorch, so the backend was not started and live AI, Qdrant, Tavily, Docker, smoke, and evaluation workflows were not executed. Historical evaluation results are reported separately from these checks.

Read Chapters 2-5 for orientation and first setup; Chapters 6-11 for ingestion, retrieval, the graph, and provider behavior; Chapters 12-15 for API, persistence, deployment, and operations; and Chapters 16-19 for evaluation, findings, and developer maintenance. Exact schema, environment, symbol, dependency, and fingerprint references are in the appendices.

2. What ScholarLens does

ScholarLens is a research assistant that retrieves text from uploaded or imported sources before answering questions. It combines a React browser interface, a FastAPI API, LangGraph orchestration, Qdrant hybrid search, a local cross-encoder reranker, configurable chat/embedding providers, and Tavily web search.

It does not train a language model. The model weights used by Ollama, the embedding provider, and the reranker are external dependencies. The project arranges retrieval, prompts, decisions, memory, and presentation around them.

Capability	Implemented behavior
Document Q&A	Load PDF, UTF-8 text, Markdown, web pages, and arXiv papers into a session's vector collection.
Hybrid search	Retrieve dense semantic and sparse BM25 candidates through Qdrant.
Reranking	Score candidate query/passage pairs with a cross-encoder; send a smaller selected set to the answer model.

Capability	Implemented behavior
Live lookup	Route current-information questions to Tavily search.
Claim checking	Search the web and arXiv-targeted results, then ask a model for a supersession verdict.
Direct answers	Answer stable general-knowledge questions without retrieval.
/btw	An off-topic answer path that bypasses vector search and LangGraph persistence.
Sessions	Separate metadata records, graph thread IDs, and Qdrant collections.
Inspection	Show retrieved excerpts and serialized graph state in the browser.
Streaming	Send answer events to the browser using an HTTP POST response with SSE framing.

2.1 Concepts from zero

Term	Meaning here
RAG	Retrieval-augmented generation: search for relevant source passages, then put them in a model prompt.
Chunk	A short piece of a source document, typically up to 1,000 characters here.
Embedding	A numeric representation used for semantic similarity search.
Dense search	Search using learned vector representations of meaning.
Sparse search	Search using lexical signals; this code uses Qdrant/bm25 .
Hybrid retrieval	Combining dense and sparse search through the Qdrant integration.
Cross-encoder	A model that jointly reads a query and passage to produce a relevance score.
Graph node	A processing step that reads graph state and returns updates.
Checkpoint	Persistence of graph state between turns, backed by SQLite.
SSE	A text event format allowing a server to send incremental updates over an HTTP response.
Golden	A question and expected answer used in evaluation.

2.2 Scope limits

There is no user login, ownership check, account-level tenancy, billing, asynchronous ingestion queue, PDF viewer, per-document deletion endpoint, model-training pipeline, or automatic fact verification beyond the configured search-and-model flow. A separate collection per session is an organizational boundary, not an authorization boundary. The API provides access to sessions without authenticating a user.

3. Repository map and architecture

rag-papeer-project-main/	
backend/	active FastAPI and RAG implementation
backend copy/	earlier dense-retrieval snapshot
backend streamlit/	same earlier snapshot; no Streamlit UI
frontend/	
src/main.jsx	complete React application
src/styles.css	dark responsive layout
package.json	Vite commands and dependencies
package-lock.json	npm resolution
nginx.conf	Docker frontend proxy
public/favicon.svg	application icon
dist/	existing built assets
node_modules/	installed third-party dependencies
documents/	sample evaluation PDF
evaluate.py	DeepEval quality evaluation
smoke_features.py	live service/feature smoke script
goldens.json	10 question/expected-answer pairs
eval_results_old.json	historical evaluation output
sessions.json	session metadata
checkpoints.db*	application SQLite state and sidecars

<code>eval_checkpoints.db*</code>	evaluation state and sidecars
<code>embedding_cache/</code>	cached embeddings
<code>.deepeval/</code>	evaluator cache and telemetry artifacts
<code>Dockerfile</code>	frontend build, backend, Nginx stages
<code>docker-compose.yml</code>	five-service stack
<code>pyproject.toml, uv.lock</code>	Python dependency resolution
<code>requirements.txt</code>	Docker pip dependency input

3.1 Runtime flow

```

Browser / React
| HTTP JSON, multipart uploads, POST SSE
v
Nginx (Compose) -> FastAPI
    |-- sessions.json: labels and IDs
    |-- SQLite: LangGraph checkpoints
    |-- document loaders -> chunks
    |
    |           |
    |           v
    |       dense + sparse embeddings
    |           |
    |           v
    |       Qdrant collection
    |
    +--> router -> selected processing path
        |-- direct model answer
        |-- hybrid search -> reranker -> answer
        |-- Tavily results -> answer
        +-- Tavily + claim analysis -> verdict

```

`backend.api` creates the graph and session-naming model at import time. Imported backend modules also instantiate provider wrappers, the Qdrant client, sparse embedding implementation, embedding cache, structured-output chains, and tool bindings. Consequently, application import has configuration and dependency requirements even before the first HTTP request. `main.py` only prints a greeting; it does not launch the API.

The source graph contains seven named nodes. A stored `graph.png` depicts its broad topology, but the active code now retrieves directly inside `agent_node`. The legacy tool node remains registered even though the normal path no longer asks the tool-bound model to issue tool calls. See Chapter 9 for the operational distinction.

3.2 The three backend folders

`backend/` is the active package imported by Uvicorn and application scripts. `backendcopy/` and `backendstreamlit/` contain eight identical Python files to each other. Six are also identical to the active backend. Their `rag_graph.py` and `vector_store.py` differ, and neither contains `reranker.py`.

The older vector store uses dense retrieval and default `k=4`; its graph explicitly requests six passages. The active implementation adds named dense/sparse vectors, default candidate `k=24`, reranking, and more conversation-aware routing/retrieval. The older collection code rejected dimension changes for non-empty collections; the active code deletes incompatible collections. The folder name `backendstreamlit` does not imply an executable Streamlit frontend. Internal imports still reference `backend`, so these folders are archival copies rather than independent selectable applications.

4. Setup from zero: choose a deployment path

The source targets Python 3.12 or newer (`.python-version` selects 3.12), a Node environment capable of building the checked-in Vite project, and access to a Qdrant service. Local model mode additionally needs Ollama and available chat/embedding models. First-use sparse and cross-encoder components may download model files.

4.1 Establish the application root

```
cd /home/punam/Music/IMPFolder/Project/just-add-reranker-aug-06-10pm-novenv
```

```
cd just-add-reranker-aug-06-10pm/rag-papeer-project-main
python3 --version
node --version
npm --version
uv --version
```

Do not print `.env` into logs or paste its contents into a support transcript. A real `.env` is present in this copy. Review configuration locally and preserve existing values rather than overwriting it with the example below.

4.2 Local Python and frontend installation

```
uv sync --locked
cd frontend
npm ci
cd ..
```

`--locked` keeps the checked-in uv resolution instead of silently updating it. `npm ci` uses the npm lockfile. The Docker path uses `requirements.txt`, not `uv.lock`; these two environments are not guaranteed identical. Several requirements use lower bounds rather than exact versions, and there is no backend build-system configuration or application console script. Uvicorn is present in the dependency resolution but is not explicitly declared as a top-level dependency in `pyproject.toml`.

If a resolved dependency cannot be installed, inspect the failure and deliberately update the relevant manifest/lockfile rather than assuming any newer version is interchangeable. This review records snapshot versions without checking current vendor availability or model catalogs.

4.3 Example local configuration

The following is a template for a new environment, not the contents of the supplied file:

```
LLM_PROVIDER=ollama
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_CHAT_MODEL=llama3.1
OLLAMA_EMBEDDING_MODEL=nomic-embed-text
OLLAMA_NUM_CTX=4096
OLLAMA_REQUEST_TIMEOUT=300
QDRANT_URL=http://localhost:6333
QDRANT_API_KEY=
QDRANT_COLLECTION_PREFIX=scholarlens
RERANKER_MODEL=BAAI/bge-reranker-base
RERANKER_MAX_DOC_CHARS=2500
RERANKER_FINAL_K=6
TAVILY_API_KEY=YOUR_TAVILY_KEY
```

Run or connect to Qdrant at the configured address. For a hosted service, use its endpoint and access key. The included `qdrant_login_steps.md` is an account-setup note whose current dashboard labels, provisioning times, and free-tier limits have not been verified. The code only requires a reachable compatible endpoint; those marketing limits are not part of the application contract.

With an installed/running Ollama service, provision the configured models:

```
ollama pull llama3.1
ollama pull nomic-embed-text
ollama list
```

Explicitly setting `OLLAMA_CHAT_MODEL` matters: backend callers pass OpenAI-style model names to the provider factory, and without the environment override those names can flow into Ollama. The factory's final `llama3.1` fallback applies only when neither an environment model nor an explicit function argument is provided.

4.4 Local same-origin serving

Build the UI and serve it through FastAPI:

```
cd frontend
```

```
npm run build
cd ..
uv run uvicorn backend.api:app --host 127.0.0.1 --port 8000
```

Open <http://127.0.0.1:8000>. The API mounts `/assets` from the built folder and returns its index document for other unmatched GET routes. A missing build returns an API-running message instead of a UI. The favicon at `/favicon.svg` is not separately mounted in this FastAPI serving path; the catch-all may return HTML for it. Nginx serves the built static files normally.

4.5 Frontend development server

The frontend defaults to same-origin `/api`, but there is no Vite proxy configuration in this tree. Merely running the API on port 8000 and Vite separately is therefore insufficient. In a second terminal, set the API base when starting Vite:

```
cd frontend
VITE_API_BASE_URL=http://127.0.0.1:8000 npm run dev
```

The backend currently allows any CORS origin without credentials. `VITE_API_BASE_URL` is a frontend build/dev-time setting; changing it in the backend container after building the UI does not rewrite the JavaScript bundle. Keep the default empty base for same-origin Nginx deployment.

5. First user workflow and UI guide

On first load, the browser requests the session list. If no sessions exist, the server creates one as a side effect of that GET. Select **Sources**, upload a small readable PDF/text/Markdown file, wait for the indexing notice, and confirm its title appears. Ask a self-contained question such as "According to the uploaded document, what method is proposed?" Inspect the retrieved passages before trusting the answer.

The source panel has three tabs. **Sources** contains file upload, multiline web URLs, arXiv input, notices, and document titles. **Retrieved** shows the selected answer's excerpts and optional links. **Graph** shows route, retrieved count, rewrite count, and expandable/copyable JSON. Reranker scores are in document metadata within that JSON; the retrieved-card UI does not separately visualize them.

The sidebar supports creating chats, title search, selecting sessions, and confirming deletion of one or all chats. The session title is generated from the first ordinary message, up to 500 characters of input, and is marked named even if naming fails and leaves "New Session." There is no manual rename endpoint.

The composer sends on Enter and inserts a line break with Shift+Enter. It grows to 180 pixels. The Stop control aborts the browser fetch and replaces local response text with "Generation stopped." There is no explicit backend cancellation protocol guaranteeing that provider computation or graph persistence stops. Reload the session to see the stored state if interrupted.

Useful example prompts are:

```
According to the uploaded paper, what are its limitations?
Who are the authors of the loaded arXiv paper?
Find recent research on retrieval-augmented generation.
Verify the claim that recurrent networks are best for translation.
/btw What is softmax?
```

"Recent research" triggers a model routing decision, not a deterministic keyword switch. `/btw` is the explicit prefix route. It is case-insensitive and currently matches any trimmed string starting with those four characters, including `/btwhatever`; a strict command-word boundary is not enforced.

6. Ingestion and document formats

6.1 Chunking

`paper_loader.py` uses `RecursiveCharacterTextSplitter` with chunk size 1,000, overlap 200, and `add_start_index=True`. These lengths are characters under the default splitter, not model tokens. Markdown uses a language-aware separator configuration with the same size/overlap. Title metadata is attached to every chunk.

Overlaps help preserve local context but create duplicate text across passages. There is no document-level deduplication or stable content-derived ID scheme. Re-uploading the same file can add more points to the collection. The document list deduplicates titles, so its count can hide repeated ingestion or distinct files with the same title.

6.2 Files

The upload API receives a multipart list named `files`. Each file is read fully into memory, written to a temporary file retaining its suffix, dispatched by extension, indexed, and unlinked in a `finally` block. The original filename stem replaces the temporary name as title metadata. PDF uses `PyMuPDF`; text and Markdown use `UTF-8 TextLoader`. Unsupported suffixes produce per-file failure records.

The endpoint returns successful titles and failed items together, generally as HTTP 200. A nonzero failure count therefore requires inspecting the JSON, not just the status code. The frontend displays counts but not the complete failure details. There is no backend upload byte limit or content-type validation; the supplied Nginx limits request bodies to 100 MB, which does not apply to direct FastAPI access.

No OCR is implemented. Scanned images inside a PDF, complex tables, and mathematical layout may not become useful text. The raw upload is not retained as a durable application source file; Qdrant stores the extracted chunks. Original `source` metadata can still refer to a deleted temporary path. A separate original-document archive is needed for reliable re-indexing.

6.3 Web URLs

The URL endpoint strips blank entries and loads each via `WebBaseLoader` with a 30-second request timeout. Extracted title metadata is used when available, otherwise the URL becomes the title. Loaded web pages are stored in the session and subsequently use vector retrieval; they are different from live Tavily search results fetched for one question.

The API's URL strings are not restricted to an approved public-host list. Server-side URL fetching can reach destinations available to the backend network. Do not expose this capability to untrusted users without network/address validation and authentication. This is a concrete gap in the implemented input boundary, not an assertion that exploitation was tested.

`WebBaseLoader` commonly provides a `source` field, while API excerpt serialization and answer prompting look specifically for `metadata.url`. The code does not normalize those fields. Consequently, some imported sources may have retrievable text but no clickable URL in the answer/source card.

6.4 arXiv

The arXiv endpoint accepts a numeric modern ID, a URL containing such an ID, or a title phrase. The regex recognizes `YYYY.NNNN` or `YYYY.NNNNN` with an optional version suffix. It strips the suffix before lookup, so asking for a specific version does not preserve that version. Old category-style IDs are not recognized by this parser.

For title search, the loader queries the arXiv Atom endpoint with `ti:"phrase"`, takes the top match, and strips its version. It fetches title/authors through the metadata endpoint, tries four PDF URL variants with 60-second timeouts, extracts pages, and chunks them. PDF title metadata takes precedence over the Atom title if populated.

When authors are available, an extra short metadata document containing title, arXiv ID, and authors is prepended. All chunks receive `arxiv_id` and, when available, `authors`. A canonical `url` field is not added. Temporary PDF files are removed. Failures become HTTP 400 in the arXiv endpoint. There is no multi-result selection UI or version-pinning record.

7. Embeddings, Qdrant, and hybrid retrieval

7.1 Embedding cache

`vector_store.py` obtains a dense embedding provider, creates `FastEmbedSparse(model_name="Qdrant/bm25")`, and wraps dense embeddings in `CacheBackedEmbeddings`. The file cache is `./embedding_cache/`, relative to the process working directory. Both document and query dense embeddings are cached. Namespace is `base_embeddings.model`; key encoding is BLAKE2b.

The cache namespace does not include provider endpoint, model revision, or dimensional configuration beyond the model name. If the same name points to changed weights or a different deployment, stale cache entries may be reused. Changing a provider/model requires an explicit cache and collection migration plan. This is a shared cache across sessions, distinct from their isolated Qdrant collections.

7.2 Collection schema

The Qdrant URL must be set: the code indexes `os.environ["QDRANT_URL"]`. Its API key is optional; empty strings become `None`. Client timeout is 45 seconds. Collection names use `QDRANT_COLLECTION_PREFIX`, default `scholarlens`, followed by the session ID with hyphens replaced by underscores.

The dense dimension is measured once per process by embedding the text "dimension probe." New collections have a named `dense` cosine vector and a named `sparse` vector with IDF modifier. `QdrantVectorStore` is constructed in hybrid retrieval mode. The exact fusion behavior is delegated to the installed integration; the repository does not explicitly configure a fusion algorithm or weights.

7.3 Destructive compatibility behavior

If an existing collection lacks the named dense vector with matching dimension or lacks a sparse vector, `get_vectorstore()` deletes it and creates a new empty collection. It does not first check that the old collection is empty, preserve documents, migrate points, or request a backup. This can happen during an add or a search. Older dense-only collections from the archived backend are incompatible.

Before switching an existing installation to this hybrid implementation, snapshot the old collections and retain original source documents. Use a new collection prefix for a deliberate re-indexing migration. A same-dimension embedding-model change can pass the schema check while making old/new vectors semantically incompatible, so schema compatibility alone is insufficient.

7.4 Search and listing behavior

The normal vector search requests 24 candidates through `similarity_search`. Search catches any exception, logs the query and error, and returns an empty list. A Qdrant outage, schema issue, or embedding failure can therefore appear downstream as no relevant evidence rather than a clear service error.

Listing scans payloads in batches of 100, collects unique title strings, and ignores vector scores. The initial collection-existence check catches service errors, but later scroll failures can still escape. Listing is performed while routing and choosing sources, so it can add noticeable latency for large collections. There is no separate durable document manifest or per-document point count exposed in the UI.

Deleting a session collection logs and suppresses Qdrant errors. The API may report the session deleted while vector data remains if the service was unavailable.

8. The reranker: what this version adds

The active backend introduces `reranker.py`. After hybrid retrieval, it scores each query/passage pair with `sentence_transformers.CrossEncoder`, sorts descending, and selects the first `RERANKER_FINAL_K` results. The default is six results from up to 24 candidates.

```
User query
-> dense + sparse retrieval: up to 24 candidate chunks
-> score each pair with CrossEncoder
-> sort by score descending
-> retain top 6
```

```
-> relevancy check sees first 3 x 300 characters
-> answer prompt receives complete selected chunk text
```

8.1 Configuration and metadata

Setting	Source default	Meaning
RERANKER_MODEL	BAAI/bge-reranker-base	Cross-encoder model name.
RERANKER_MAX_DOC_CHARS	2500	Passage characters passed to the scoring model.
RERANKER_FINAL_K	6	Number of sorted candidates retained.

The README example specifies `BAAI/bge-reranker-v2-m3`, but the code fallback is the base model. A bare model name receives a `BAAI/` prefix; blank input falls back to the base model. Model configuration is read at module import, so restart the process after changing it.

The encoder is loaded lazily on the first nonempty candidate list. Empty retrieval returns immediately. There is no explicit device, inference batch size, model revision, loading lock, download timeout, or exception fallback in this module. Hardware choice and token truncation beyond the explicit character slice depend on Sentence Transformers. The first retrieval can be much slower than subsequent ones, and local inference resources are needed even when the answer LLM is remote.

Selected documents are newly constructed with their original full text and metadata plus `rerank_score`, `rerank_rank`, and `retrieval_candidate_rank`. The score is a ranking output, not a calibrated probability or confidence in answer truth. No score threshold filters weak passages; top-N selection can still return irrelevant content. The later LLM relevance check is the separate gate.

`top_n` and the integer environment settings are not validated for useful positive values. Zero selects none; negative slicing has ordinary Python semantics. Keep them positive and evaluate changes on held-out questions. A larger final set increases answer context and latency, and can exceed a small Ollama context window.

8.2 What reranking does not affect

Normal Tavily retrieval and claim verification are not reranked. The legacy `retrieve_from_vectorstore` tool calls vector search directly with its tool `k` and does not call the reranker. It is not the normal active path, but a future reactivation of tool-based retrieval could bypass reranking unless explicitly updated.

The historical `eval_results_old.json` cannot establish a reranking improvement because it has no trustworthy pairing to a current reranker baseline or code revision. Compare dense-only, hybrid-only, and hybrid-plus-reranker runs on the same corpus/questions before making such a claim.

9. LangGraph routing and state

9.1 State fields

`RAGState` extends `MessagesState`. In addition to accumulated messages, it tracks `session_id`, `query`, `route`, `retrieval_source`, `retrieved_docs`, `retrieval_attempts`, `claim_verdict`, `claim_source`, `superseding_papers`, `answer`, `is_relevant`, and `rewrite_count`. The API resets transient fields at the start of an ordinary turn but reuses the session ID as the graph thread ID, retaining message history in SQLite.

`_format_recent_messages()` takes the last eight message objects by default, keeps human/assistant messages, collapses whitespace, and truncates each content string to 800 characters. This is a bounded prompt window, not a conversation summarizer. Tool messages are skipped after the initial last-N slice.

9.2 Router

The router asks a structured-output model to choose `retrieve`, `verify_claim`, or `direct_answer`. Its prompt contains current-session document titles and recent conversation. Loaded-source questions and current

information should route to retrieval; explicit supersession/finding checks should route to verification; stable self-contained questions should route directly.

The router relies on model output and the Pydantic literal schema. There is no deterministic fallback route for schema or provider failure. The active query is initially the user's message; router output changes only **route**.

9.3 Retrieval source selection

agent_node lists sources and uses another structured decision for **vectorstore** versus **web_search**, unless a source is already retained from a rewrite. Vectorstore mode expands queries of three words or fewer with up to six recent messages and a "Latest question" label, then retrieves candidates and reranks them. Longer queries do not receive that explicit expansion.

Web mode calls `_search_web_documents(query,max_results=5)` directly. Results become **Document** objects with snippet content, title, and URL. They are transient graph context rather than newly indexed papers. Errors are wrapped in a user-facing service-unavailable message and ultimately emitted as SSE errors.

9.4 Relevance and rewrite

The relevance node sees only the first three selected documents, each truncated to 300 characters, along with recent conversation and the current query. The prompt is deliberately lenient: substantive partial overlap should count as relevant. The decision schema contains a reason, but the graph only stores its Boolean.

If irrelevant, the rewrite node asks the model for a better query, clears retrieved documents, resets attempts to zero, increments **rewrite_count**, preserves the chosen retrieval source, and appends the rewritten query as another **HumanMessage**. There is at most one rewrite under the normal route, so the typical path is one retrieval or two retrievals, not three despite **MAX_RETRIEVAL_ATTEMPTS=3**.

The rewrite does not switch from uploaded sources to live web or vice versa. On a second irrelevant result, answer generation emits a fixed inability-to-find-evidence response. Appending an internal rewrite as a human message means it can appear in reloaded chat history as if the user wrote it.

9.5 Legacy tool wiring

The graph still defines retrieval tools, a tool-bound **retrieval_llm**, a **ToolNode**, and conditional edges supporting tool messages. However, the current **agent_node** invokes source selection and retrieval functions directly; it never invokes **retrieval_llm**. The prompt **RETRIEVE_SYSTEM** is also retained without driving that direct path.

Do not describe this version as a model freely choosing arbitrary k and tool queries on each retrieval turn. In the normal active path, vector candidate k is 24, final k is configured by the reranker, and web results are five. Tool schemas still advertise k/result values in the 1-10 range, but those are legacy-tool interfaces, not the controlling settings for ordinary retrieval.

9.6 Graph persistence

`build_graph(db_path="checkpoints.db")` opens SQLite with `check_same_thread=False`, creates a **SqliteSaver**, registers seven nodes, sets router as entry, adds conditional edges, and compiles the graph. The SQLite connection is not explicitly closed through a FastAPI lifespan handler. The path is relative to process CWD. Use the application root consistently and avoid assuming multiple workers are safe merely because cross-thread access is enabled.

10. Answer generation, web search, and claim verification

10.1 Retrieved answers

The answer prompt includes each selected document's title, optional **url**, and full selected chunk text. It asks the model to use only that retrieved context and cite source titles or URLs when helpful. There is no token-budget trimming over the concatenated context, no mandatory citation schema, and no programmatic validation that each answer claim is supported.

Although conversation history is computed in `generate_answer_node`, the active retrieved-answer prompt does not include it. A more conversation-aware prompt exists only as commented code. Short follow-ups may improve through expanded retrieval, but final answer generation still sees the current or rewritten query and context rather than full conversational meaning.

When no documents are present, the answer is "I don't know the answer." After an unsuccessful rewrite, the graph returns a more specific source/web failure message. Otherwise it invokes the answer LLM and appends an `AIMessage` with the final answer.

10.2 Direct answers

The direct prompt identifies the assistant as ScholarLens, includes recent conversation, and asks it to resolve short follow-ups or ask a concise clarification when necessary. It answers from model knowledge. It is not grounded in the vector store or a fresh web search. The graph inspector's route helps distinguish this from a retrieved response.

10.3 Claim verification

The verifier makes two Tavily requests: `recentresearchsuperseding:<first200claimcharacters>` and `site:arxiv.org<first200claimcharacters>`, each with up to five results. Titles, URLs, and snippets up to 300 characters are passed to a structured-output model. The prompt asks whether the claim has been superseded/challenged/updated and requests up to three superseding papers.

There is no explicit publication-date filter or check that a cited work is newer than the original paper. "Recent" is part of the search query text. The claim is the user's message rather than an automatically extracted statement from an uploaded source. Search-result titles/URLs are constrained by prompt wording but not checked against the returned structured values.

The schema contains `is_superseded`, `verdict_summary`, and a list of title/URL/summary records. The graph stores the summary, first returned URL, and first three records; it drops `is_superseded`. Display logic is based on whether the paper list is nonempty. Empty results produce "No papers directly superseding this claim were found," which should not be interpreted as proof that the claim remains valid.

The final verdict is formatted in Python, so it may arrive through a `replace/final` event without ordinary answer-model token streaming. Search context is not retained as `retrieved_docs` in this route, limiting the evidence exposed by the Retrieved tab.

10.4 ‘/btw’ side channel

The side handler first asks a model whether live search is needed. If so, it fetches three Tavily results and adds their snippets/URLs to the answer prompt; otherwise it answers from general knowledge. Its answer chain streams text directly. It does not touch the graph checkpointer or vector store and receives no ordinary session conversation history.

The API returns `saved:false`. The browser still displays the local user/assistant messages until the session is reloaded, but does not add an explicit unsaved badge. External providers may still process the query; "not saved in session history" is not equivalent to "never transmitted." An empty `/btw` query returns an SSE error and no final event.

11. Provider selection and configuration precedence

11.1 Chat provider

`LLM_PROVIDER=openai`, `groq`, or `ollama` explicitly selects a provider. Otherwise, the factory chooses OpenAI if its key exists, then Groq if its key exists and no OpenAI key exists, otherwise Ollama. Unrecognized provider strings fall through to the same key-based logic rather than being rejected.

Provider	Model-selection order in code	Embedding provider
OpenAI	Explicit function argument, then <code>OPENAI_CHAT_MODEL</code> , then <code>gpt-5-mini</code>	OpenAI embeddings
Groq	<code>GROQ_CHAT_MODEL</code> , then explicit argument, then <code>llama-3.1-70b-versatile</code>	Ollama embeddings
Ollama	<code>OLLAMA_CHAT_MODEL</code> , then explicit argument, then <code>llama3.1</code>	Ollama embeddings

The graph explicitly calls `get_chat_model("gpt-5.4-mini")`; naming and /btw explicitly call `get_chat_model("gpt-5-mini")`. For OpenAI mode, these arguments override `OPENAI_CHAT_MODEL`. For Groq/Ollama, set the appropriate environment model to avoid forwarding those provider-specific names. These are the literal identifiers in this snapshot, not verified claims about current vendor support.

Groq uses `ChatOpenAI` with a configurable OpenAI-compatible base URL. It has no native Groq embedding path. Choosing Groq for chat still requires a reachable Ollama embedding model under the current implementation. A pure remote-Groq/no-Ollama setup is therefore incomplete.

11.2 Ollama hardware settings

Ollama's context defaults to 4,096 in code and Compose; the README example sets 2,048. Request timeout defaults to 300 seconds. If `OLLAMA_NUM_GPU` is unset, no explicit value is passed. If numeric, it is parsed as an integer. If `auto`, the code asks local `nvidia-smi` for free GPU memory and forces CPU (`num_gpu=0`) below a default threshold of 5,000 MB; otherwise it leaves Ollama to choose.

That probe runs where the Python process runs, not necessarily on the host of a remote Ollama server. In the supplied backend container, `nvidia-smi` is not explicitly installed and the backend has no GPU assignment. Probe failure returns no override. Compose instead defaults to `OLLAMA_NUM_GPU=999` and assigns GPUs to the Ollama service. The local reranker is in the backend process, so it has a different device environment from Ollama.

11.3 Structured output

The wrapper first calls `model.with_structured_output(schema)`. Only a `NotImplementedError` during that construction triggers the fallback: bind JSON format, extract content, strip a code fence, parse JSON, and validate through Pydantic. Runtime API errors or malformed native structured output do not automatically trigger another provider method. No generic retry/repair loop is implemented by this wrapper.

The fallback schemas enforce allowed route/source strings and record types, but not truthfulness or source-link existence. Imported libraries may have their own retries; this manual does not infer those defaults as explicit project guarantees.

12. HTTP API and streaming contract

The API title is ScholarLens API, version 0.2.0. FastAPI's generated schema and interactive docs normally live at `/openapi.json` and `/docs` on the backend. The Nginx configuration proxies only `/api/`; those documentation routes are not exposed as backend routes through the frontend proxy by default.

12.1 Endpoint overview

Method / path	Behavior
GET <code>/api/health</code>	Returns <code>{"status":"ok"}</code> ; no dependency probes.
GET <code>/api/sessions</code>	Lists newest-first metadata; creates a session when empty.
POST <code>/api/sessions</code>	Creates UUID session metadata.
DELETE <code>/api/sessions</code>	Deletes listed vector collections and clears metadata.
GET <code>/api/sessions/{session_id}</code>	Returns metadata, reconstructed messages, and source titles.
DELETE <code>/api/sessions/{session_id}</code>	Removes metadata and attempts vector deletion; returns next ID.
GET <code>/api/sessions/{session_id}/documents</code>	Lists unique indexed titles.
POST <code>/api/sessions/{session_id}/documents/files</code>	Multipart upload and inline indexing.
POST <code>/api/sessions/{session_id}/documents/urls</code>	JSON <code>urls</code> list; inline page loading/indexing.
POST <code>/api/sessions/{session_id}/documents/arxiv</code>	JSON <code>query</code> ; inline paper loading/indexing.
POST <code>/api/sessions/{session_id}/chat</code>	JSON <code>message</code> ; returns an SSE stream.
GET <code>/[{path:path}]</code>	Frontend index fallback or API-running message.

There are no ownership headers, session cookies, or authorization dependencies. An unknown session ID produces 404 for the session-scoped operations. Empty chat input produces 400. Missing/wrong request fields are subject to Pydantic/FastAPI request validation. File/URL per-item errors are returned in `failed`, while arXiv failures produce HTTP 400.

12.2 Minimal API exercise

Run against an intentionally selected local development instance after its dependencies are ready. These examples create a session and index a supplied file; they were not executed during the documentation review.

```
curl -sS http://127.0.0.1:8000/api/health
curl -sS -X POST http://127.0.0.1:8000/api/sessions
```

Copy the returned session ID into a task-specific shell variable:

```
SCHOLAR_SESSION_ID='REPLACE_WITH_RETURNED_ID'
curl -sS -X POST \
  "http://127.0.0.1:8000/api/sessions/$SCHOLAR_SESSION_ID/documents/files" \
  -F 'files=@documents/Openclaw_Research_Report.pdf'
curl -N -X POST \
  "http://127.0.0.1:8000/api/sessions/$SCHOLAR_SESSION_ID/chat" \
  -H 'Content-Type: application/json' \
  -d '{"message":"According to the report, what limitations are discussed?"}'
```

`curl -N` disables client buffering for stream observation. A browser's native `EventSource` is not used because this endpoint accepts POST with a JSON body; the frontend reads the fetch response stream manually.

12.3 SSE events

Each event is `event:<name>`, followed by one `JSON data:` line and a blank line. Unicode is emitted without forced ASCII escaping.

Event	Payload fields and meaning
<code>start</code>	<code>mode</code> : <code>graph</code> or <code>btw</code> .
<code>token</code>	<code>content</code> : append this text to the visible answer.
<code>replace</code>	<code>content</code> : replace the full accumulated answer.
<code>final</code>	<code>answer</code> , <code>graph_state</code> , <code>retrieved_context</code> , <code>saved</code> ; normal graph replies also include <code>session</code> .
<code>error</code>	<code>message</code> : terminal error text for this stream.

The normal API exposes only message chunks whose LangGraph metadata identifies `generate_answer`. Router, selector, relevance, and verification reasoning are not streamed as answer text. The graph uses model `.invoke()` inside nodes; token events depend on the integration's message-stream behavior. A final state answer that differs from accumulated tokens is sent as `replace` before `final`.

Streaming errors occur after HTTP headers may already be sent, so an HTTP 200 does not guarantee success. Clients must consume the events. There is no heartbeat, reconnect ID, replay cursor, or resumable stream protocol. The frontend splits on `\n\n`; it does not flush an unfinished trailing block or explicitly require a final event before treating EOF as finished. A cleanly closed but incomplete stream may leave an answer marked pending.

12.4 State and excerpt serialization

Graph messages and retrieved document contents are truncated to 300 characters in the serialized graph state, while retrieved-context cards receive up to 1,500 characters plus full metadata. Other state values, including the answer, are passed through. This is a debug snapshot rather than a full downloadable source record. Metadata may expose original source paths; do not treat the inspector as a redacted public audit report.

13. Session memory, persistence, and recovery

13.1 Three different stores

Session metadata is stored in root-relative `sessions.json` with `id`, `name`, `created_at`, and `is_named`. Conversation/graph checkpoints use CWD-relative SQLite `checkpoints.db`. Indexed source passages use remote/local Qdrant collections. Dense embeddings use another CWD-relative file cache. These stores are related by IDs, but there is no transaction spanning them.

The supplied snapshot contains one session metadata record, application and evaluation SQLite databases with `checkpoints` and `writes` tables, and 2,302 embedding-cache files. Those artifacts do not prove that the configured Qdrant endpoint still contains corresponding collections or that a stored response was generated by the current source. Private chat contents are not copied into the manual.

13.2 Persistence limitations

`save_sessions()` rewrites the entire JSON file without locking or atomic replacement. Concurrent create/rename/delete operations can overwrite one another. Invalid JSON causes `load_sessions()` to return an empty dictionary; a later list/create can replace recoverable metadata with a new session set. There is no versioned metadata backup.

Deleting a session removes metadata and attempts Qdrant deletion, but does not delete its SQLite graph checkpoints or embedding cache. "Delete all chats" is therefore not complete data erasure. The browser immediately refreshes the empty session list, which creates a new empty session. If Qdrant deletion fails, it is logged and suppressed.

On reload, `load_session_chats()` reads only the latest graph state. It attaches that latest graph snapshot and current retrieved documents to the first reconstructed assistant turn and attaches empty snapshots to later

turns. It does not reconstruct per-turn evidence. This is a source-confirmed mismatch with the README's implication that each historical answer has its own inspectable graph state. Live answers do initially retain their final event data in browser memory.

13.3 Backup and restore procedure

For an intentional backup, stop writes or use service-native consistent snapshots. Preserve session metadata, the SQLite database with its necessary journal/WAL state, Qdrant collection snapshots, original source files, configuration provenance, and model identifiers. Embedding cache can be preserved for speed but should not be the only source of truth. Credentials should be backed up through an appropriate private mechanism, not included in a distributable project archive.

Copying an active SQLite main file alone can omit uncheckpointed WAL content. Compose binds only `checkpoints.db`, not its sibling WAL/SHM paths; inspect the actual journal behavior and storage before relying on container recreation as a recovery procedure. The supplied snapshot already contains SQLite sidecars. A dedicated persistent directory for related state would be easier to operate than several unrelated root files.

Restore compatible collection schemas and embedding models before resuming document search, because the current schema-repair code may delete mismatched collections. Confirm session IDs align across metadata, graph threads, and Qdrant. Health status alone is not a restore test; open a session, inspect its sources, and retrieve a known passage.

14. Docker and deployment

14.1 Build stages

The Dockerfile has three named stages: `frontend-build` uses Node 20 and `npmci`; `runtime` uses Python 3.12, installs `requirements.txt`, copies active backend/sample data/metadata and built UI, and runs Uvicorn on port 8000; `frontend-runtime` uses Nginx, copies static UI and proxy configuration, and exposes port 80.

Without `--target`, Docker builds the last stage, `frontend-runtime`. The existing guide's generic `docker build -t scholarlens:latest` followed by `-p8000:8000` therefore does not produce the described API image. It produces Nginx expecting a backend hostname. For an intentionally standalone API image, select the stage explicitly:

```
docker build --target runtime -t scholarlens-backend:manual .
```

Running that image still requires reachable configured services and deliberate persistence mounts. `localhost` inside the container is the container itself, not the host Ollama/Qdrant service. The Dockerfile copies the supplied `sessions.json`, so existing metadata is baked into the backend image unless intentionally prepared differently. The build does not provision chat, embedding, BM25, or cross-encoder model weights.

14.2 Compose services and ports

Service	Purpose	Connectivity / persistence
<code>ollama</code>	Local model runtime with <code>gpus:all</code>	Internal 11434; named model-data volume.
<code>ollama-pull</code>	Intended one-shot provisioning	Depends on Ollama health; shares model volume.
<code>qdrant</code>	Local vector DB	Internal 6333; named storage volume.
<code>backend</code>	FastAPI and retrieval pipeline	Internal 8000; binds sessions, SQLite file, embedding cache.
<code>frontend</code>	Nginx static UI and API proxy	Host <code>SCHOLARLENS_PORT</code> , default 8020, to container 80.

Compose explicitly overrides backend Qdrant URL to `http://qdrant:6333`, API key to empty, and Ollama URL to `http://ollama:11434`. It therefore uses internal services even if `.env` contains a cloud Qdrant address. GPU assignment is to Ollama, not the backend reranker. Selecting OpenAI/Groq does not remove the existing Ollama service/dependency chain or its GPU requirement.

14.3 Model-pull blocker and recovery route

The actual `ollama-pull.entrypoint` is `/bin/sh, -c,` followed by an empty multiline scalar. It runs no model-pull commands. The backend waits for that service to complete successfully, but a successful empty shell does not mean required models exist. This is a source-confirmed setup defect.

A manual provisioning route, without pretending the empty service works, is:

```
docker compose up -d ollama qdrant
docker compose exec ollama ollama pull llama3.1
docker compose exec ollama ollama pull nomic-embed-text
docker compose build
docker compose up -d backend frontend
docker compose logs -f backend frontend
```

Use the actual configured model names rather than these example names if overridden. The commands require a prepared `.env`, compatible GPU container runtime under this Compose file, and correct bind-mount files. A new checkout must ensure `sessions.json` and the database bind source are files rather than accidentally created directories. This documentation review does not start or repair the stack.

An alternative code fix is to populate the init service's shell body with model pulls, use failure-sensitive shell execution, and verify completion before backend startup. That is a proposed implementation change; the original Compose file remains unchanged.

14.4 Nginx streaming and limits

Nginx proxies `/api/` to `scholarlens-backend:8000`, uses HTTP/1.1, disables proxy buffering, and sets 300-second read/send timeouts. It forwards host/client/protocol headers. Assets receive a one-year immutable cache policy; other frontend paths fall back to index HTML. The request body limit is 100 MB.

There is no TLS termination or authentication layer in this configuration. Backend health returns only a constant status and may pass while models or vector services are unavailable. Qdrant is gated only by container start, not health. Image tags include `latest`, and Python requirements are partly unpinned, so two builds can differ without source changes.

The included AWS text file installs Docker on Ubuntu; it does not configure GPU container support, TLS, authentication, backup, or application readiness. Treat it as historical setup notes, not a complete production deployment runbook.

15. Frontend internals and operating behavior

15.1 Component structure

`main.jsx` contains all UI state and components. `App` owns session selection, messages, documents, input, loading, panel state, notifications, last prompt, abort controller, scrolling, and resize handling. Child components are `Sidebar`, `TopBar`, `ConversationViewport`, `EmptyState`, `ChatMessage`, `Composer`, `SourcePanel`, `Notice`, `RetrievedContext`, and `GraphState`.

Helpers implement class-name joining, friendly errors, JSON API fetches, clipboard copy with a DOM fallback, localized dates, and SSE block parsing. React Markdown renders GFM tables and math through Remark/KaTeX. The current code does not enable raw HTML through a rehype-raw plugin. Some imported icons are unused; no lint/typecheck/test script is configured in `package.json`.

15.2 State and stream handling

`sendPrompt()` appends optimistic user and empty assistant entries using browser UUIDs, creates an `AbortController`, and reads a UTF-8 response stream. Token events append, replace events replace, final events attach

graph/context data and refresh session labels, and error events mark the assistant failed. Retry sends the last prompt again; it is not an idempotent continuation of a failed request and can create another stored user turn. Only one send is allowed at a time by the global loading flag. The sidebar still allows selecting another session or creating one during an in-flight stream. Effects and requests have no per-session cancellation or stale-response guard. A late load or stream completion can therefore update the wrong visible session or switch focus unexpectedly. This is an inferred race from the code, not a browser-reproduced failure in this review.

Abort replaces local partial text rather than preserving it. Network error classification turns several error patterns into friendly messages. A stream that ends without `final` lacks explicit incomplete-response handling. Browser state should not be treated as authoritative persistence after interruption.

15.3 Layout and accessibility boundaries

The CSS defines a dark palette, 276-pixel desktop sidebar, 360-pixel default source panel, 58-pixel top bar, bounded conversation width, scrollable message/source panels, and a fixed-height viewport shell. Desktop source resizing is clamped to 320-760 pixels subject to viewport width, and saved in `localStorage` under `scholarlens-source-panel-width`.

At 1,200 pixels and wider the main pane leaves space for the source panel; narrower layouts use an overlay/scrim. At 1,024 pixels the sidebar narrows and suggested prompts become a single column. At 768 pixels the sidebar becomes a drawer, the resize handle disappears, and the source panel is capped at 94vw/390px. At 390 pixels avatars disappear and typography compacts.

Buttons include several accessible labels and panels have named regions. However, source tabs lack complete tab keyboard semantics, the resize handle has no keyboard resizing logic, and no focus-trap/Escape-dismiss implementation is present. No accessibility audit or responsive browser test was performed as part of this documentation review.

16. Evaluation, sample data, and smoke checks

16.1 Included evaluation fixture

`documents/Openclaw_Research_Report.pdf` is a 34-page report used as evaluation input; its internal title is a Moltbot comprehensive analysis report. Its claims are fixture content, not facts verified by this manual. `goldens.json` contains ten input/expected-output pairs. Existing goldens are reused; if absent, a DeepEval synthesizer creates examples from up to five contexts and two goldens per context and writes the file.

The synthesizer is constructed without the provider wrapper or the custom judge model, so its provider behavior is delegated to DeepEval defaults. Do not assume that choosing Ollama for application chat makes synthetic-golden generation entirely local.

16.2 Evaluation execution

```
uv run python evaluate.py
```

This is a live workload: it loads the PDF, creates an evaluation graph at `eval_checkpoints.db`, creates one unique Qdrant session collection per golden, indexes the same document for each, invokes the graph, and evaluates the answer/context. The actual query receives "as per the report in knowledge base," encouraging retrieval. Judge input records the original golden question.

Metrics are contextual precision, contextual recall, contextual relevancy, answer relevancy, and faithfulness, all at threshold 0.7. Reasons are disabled, metric async mode is false, and evaluation concurrency is explicitly serialized. In Ollama mode, the judge uses `OLLAMA_CHAT_MODEL` with a `gemma3` fallback, context 4,096 and prediction limit 1,024; other modes use the literal judge name `gpt-5.4-mini`. Groq application mode does not imply a Groq judge.

Summary output is `eval_results.json`: question, actual answer, overall success, and each metric's name/score/pass/reason. Temporary Qdrant collections are deleted in a `finally` loop, but evaluation SQLite state remains. Cleanup errors can interrupt the cleanup loop. This workload can consume paid service calls and should be run deliberately against designated evaluation storage.

16.3 Historical results in this copy

The following arithmetic summarizes `eval_results_old.json`; it is not a fresh result and is not evidence that the current hybrid/reranking version achieved these scores.

Historical metric	Mean across 10 records
Contextual Precision	0.9783
Contextual Recall	1.0000
Contextual Relevancy	0.5756
Answer Relevancy	0.9909
Faithfulness	0.9809

Five of ten records have overall `success=true`. The result file does not establish a code fingerprint, reranker setting, model revision, or a controlled comparison. The lower contextual relevancy alongside high recall can motivate retrieval investigation, but it does not by itself diagnose the cause or prove the new reranker fixes it.

16.4 Live smoke script

`smoke_features.py` checks environment presence, chat and embeddings, Qdrant connectivity/add/list/search, Tavily, text/web/arXiv loaders, both `/btw` modes, legacy web-tool invocation, graph direct/retrieve/web/verification paths, and rewriting. It writes JSON summary to stdout, returns nonzero if any recorded check fails, and tries to clean its temporary Qdrant collection and SQLite files.

It is not an offline unit-test suite. It uses actual credentials, external requests, model calls, and vector writes. Its environment-presence check requires `QDRANT_API_KEY` even though the local Compose mode intentionally leaves it empty; that can fail in a valid local configuration. Some labels say "ollama" even if another provider is active, and several checks verify only nonempty outputs rather than answer correctness. It does not directly cover API authorization, SSE completion, browser race conditions, reranker ranking quality, or session deletion retention.

16.5 Validation performed for this manual

All 28 original Python files passed syntax parsing. The frontend built successfully with 1,859 transformed modules using the installed dependencies; Vite warned about a JavaScript chunk larger than 500 kB. Build output went to `/tmp/scholarlens-documentation-build`, preserving the supplied `frontend/dist`.

Eleven offline checks covered syntax, model-name normalization, empty reranking, scoring-text truncation, descending/top-N selection, rank metadata, nonmutation of candidate metadata, arXiv version stripping/legacy-ID behavior, and SSE framing. These checks extracted source functions and used a lightweight document stub and fake encoder. They do not test real Sentence Transformers inference or application dependency compatibility. Results and the reproducible validation script are included beside this manual.

The backend was not imported or started because its ML/API dependencies are absent in the available Python interpreter. Live smoke/evaluation were not run. No current-system Qdrant collections, cloud accounts, or model services were modified by this documentation task.

17. Troubleshooting and operating checklist

Symptom	Likely explanation / next step
<code>pythonmain.py</code> only prints a greeting	Start <code>uvicornbackend.api:app</code> ; <code>main.py</code> is not the server.
Missing Qdrant variable during import	Set <code>QDRANT_URL</code> in the application environment before import.
Vite UI cannot load sessions	Set <code>VITE_API_BASE_URL</code> for separate-origin development; there is no Vite proxy.
Health passes but chat fails	Health does not probe models, keys, Qdrant, or Tavily.
Ollama requests an OpenAI-style name	Set <code>OLLAMA_CHAT_MODEL</code> ; explicit factory arguments can override the fallback.

Symptom	Likely explanation / next step
Groq chat works but ingestion fails	Embeddings still use Ollama in Groq mode.
Compose starts with missing models	The model-pull endpoint is empty; provision models explicitly.
Single Docker image has no API on 8000	Default build target is Nginx; select runtime for the backend.
Documents disappear after an upgrade	Hybrid schema mismatch can delete/recreate collections; restore/re-index deliberately.
No evidence instead of a connection error	Vector search catches failures and returns an empty list; inspect backend logs.
First retrieved answer is very slow	Lazy cross-encoder loading and model download may occur.
Imported page has no Open link	source metadata is not normalized to the url field the UI reads.
Scanned PDF produces weak/empty context	No OCR stage exists.
Re-upload count looks unchanged	Listing deduplicates titles, not vector points.
Old answer shows wrong graph context	Reload attaches latest state to the first assistant turn; per-turn replay is incomplete.
Stop does not stop model resource use	Browser abort is not a provider cancellation protocol.
Deleted chats remain in database	SQLite checkpoints and embedding cache are not removed by deletion APIs.
Smoke fails only API-key presence	Local Qdrant can legitimately have an empty key; smoke assumes one is present.

Before first use, verify the correct working directory, dependency environment, model names, Qdrant schema/migration plan, available disk/RAM, and whether the reranker can load. Start with one small document and a known factual question. Verify both retrieved text and the final answer, then test session switching and `/btw` behavior.

During operation, monitor request failures, first-token and total latency, model memory use, Qdrant availability, ingestion count/size, SQLite/cache growth, and the rate of no-evidence responses. Avoid interpreting empty-source lists as certain data loss until service errors have been ruled out. Preserve a repeatable small evaluation set before changing chunking, embeddings, prompts, reranker, or model providers.

No automatic scheduled backups, metrics dashboard, tracing setup, rate limiting, task queue, or cost accounting is configured by the application source. Imported telemetry/tracing libraries do not alone constitute an implemented observability system.

18. Review findings and prioritized improvements

These are source-based findings unless explicitly covered by offline validation. This task adds documentation and review helpers; it does not change the application to fix these issues.

18.1 Deployment and data integrity

- The Compose model-pull service runs an empty command; readiness does not ensure model availability.
- The image-only guide selects the last Docker stage implicitly and publishes the wrong internal port for its described API behavior.
- The local development guide omits the separate-origin API base setting required by the actual frontend.
- Collection compatibility repair deletes existing data without a migration path, including populated dense-only collections.
- Session JSON writes are non-atomic and unlocked; invalid JSON silently appears as an empty session set.
- Deleting sessions does not remove graph checkpoints or shared embedding cache, and vector deletion failures are suppressed.

- Provider model precedence and Groq's dependence on Ollama embeddings can make seemingly valid configurations fail.
- The Docker image copies existing session metadata, while backups and journal sidecars are not comprehensively managed.

18.2 Public-service boundary

The API has no authentication or session ownership checks and permissive CORS. URL fetching lacks destination validation, file uploads are read into memory without application-level limits, raw errors can expose internal details, and debug metadata can expose source paths. These concrete behaviors make the current implementation more appropriate for a controlled environment than an unmodified public multi-user service.

Recommended hardening is to add identity/ownership enforcement, network-safe URL validation, upload/request limits, controlled error serialization, consistent state deletion, and a deliberate TLS/reverse-proxy policy. Retrieved source text is embedded directly into model prompts; there is no implemented prompt-injection boundary or independent citation verifier. Do not equate "use only context" prompt wording with enforced grounding.

18.3 Retrieval, history, and quality

- Only the first 900 characters across three chunks determine the relevance decision, even though more context may be available.
- The normal retrieved-answer prompt omits conversation history despite history-aware routing.
- Query rewrite overwrites the working query and appends an internal human message to persisted history.
- Source choice is retained through the rewrite, so there is no automatic cross-source recovery.
- Legacy tool definitions remain but are not the normal active retrieval path; comments and configuration can mislead maintainers.
- Reranker defaults differ between README and code; no explicit scoring threshold, model revision, or device setting is supplied.
- Embedding cache identity relies on model name alone; same-dimension model changes can evade collection compatibility checks.
- arXiv version suffixes are discarded and imported source URLs are incompletely normalized.
- Claim supersession output lacks date/order validation and returned-link verification, and drops the Boolean result from state.
- Historical per-turn graph/context inspection is reconstructed incorrectly from only the latest snapshot.

18.4 Frontend and validation gaps

Session switching and creation are not isolated from in-flight requests. EOF without a final SSE event is not explicitly handled. Retrying is not idempotent, and stop semantics are only client-side. There is no checked-in frontend test/lint script, and the live smoke runner is not a substitute for API and deterministic unit tests. The old evaluation file is not a benchmark of this revision.

Recommended order: first protect collection/state integrity and repair startup instructions; then make provider/-model configuration explicit and stable; then add deterministic API/stream/session tests; then create controlled retrieval/reranker evaluations; finally add multi-user security and production operations if public deployment is required.

19. Developer maintenance and extension guide

19.1 File responsibility map

File	Responsibility
<code>backend/api.py</code>	Request schemas, sessions, uploads, SSE transport, graph-state serialization, frontend fallback.
<code>backend/paper_loader.py</code>	Format dispatch, text extraction, chunking, arXiv metadata/downloads.
<code>backend/vector_store.py</code>	Dense/sparse embeddings, cache, collection schema, add/list/search/delete.
<code>backend/reranker.py</code>	Lazy cross-encoder scoring and selected-document rank metadata.
<code>backend/rag_graph.py</code>	State, prompts, decisions, search/verification/generation nodes, edges, SQLite saver.
<code>backend/providers.py</code>	Chat/embedding provider selection, Ollama options, structured-output wrapper.
<code>backend/models.py</code>	Structured routing, relevancy, source, and verification schemas.
<code>backend/btw_handler.py</code>	Stateless off-topic routing and answer stream.
<code>frontend/src/main.jsx</code>	All React components, fetch/SSE logic, and UI state.
<code>frontend/src/styles.css</code>	Visual tokens, responsive layout, panels, messages, Markdown/math styling.
<code>evaluate.py</code>	Golden generation/reuse, RAG cases, judge metrics, result export.
<code>smoke_features.py</code>	Live feature checks and temporary-artifact cleanup.

The appendices locate every Python function/class, frontend named function, request/state schema, and referenced environment setting. Archived backend differences are explicitly listed rather than treated as active modules.

19.2 Making controlled changes

For a new loader, return `LangChain Document` objects with consistent `title`, `source`, canonical `url` when applicable, and provenance metadata. Add it to dispatch/API/UI deliberately and test extraction failures and partial upload results. Keep original documents if re-indexing matters.

For a new embedding model, select a new collection namespace, record model identity/dimension, regenerate embeddings, and validate retrieval against known questions before cutting over. Avoid relying on implicit schema repair. For reranker changes, record candidate `k`, final `k`, character limits, model revision/device, latency, and ranking quality with the same candidate set.

For graph changes, keep transient per-turn state reset in sync with API and evaluation callers, preserve tool-message matching if reactivating `ToolNode`, and test the rewrite/abstention paths. Separate internal rewritten search text from user-visible history. For streaming changes, version or document event semantics and test Unicode chunks, malformed/error events, interrupted EOF, and cancellation.

For frontend maintenance, split the large component module only when it improves responsibility boundaries, preserve source-panel resize cleanup, and add stale-response guards keyed by session and request ID. For deployment, consolidate dependency resolution and make startup/readiness explicitly validate the selected model and vector services.

19.3 Rebuilding this manual

The documentation is stored in the outer wrapper's `docs/` directory, outside the application source tree. `manual.md` is the authored narrative. `build_manual.py` derives appendices, writes a complete Markdown manual, creates source fingerprints, and compiles the PDF through `render_pdf.py` and local LaTeX. It does not import backend modules or read secret values for publication.

```
# From just-add-reranker-aug-06-10pm-novenv:
python3 docs/validate_source.py
python3 docs/build_manual.py
```

The complete Markdown and JSON references remain editable. Automatic regeneration keeps line indices/default tables current, but a human still needs to update the narrative when behavior changes. Source hashes identify this reviewed snapshot; they are not evidence of deployment identity or historical benchmark provenance.

A. Exact API endpoint and request reference

Extracted from decorators and schema definitions in the active backend. The API table in Chapter 12 explains semantics; line locations below identify the implementation. FastAPI-generated documentation routes are not explicit decorators in this source.

A. GET /api/health

Handler: `health`; source `backend/api.py:189`.

Direct JSON response keys: `status`.

A. GET /api/sessions

Handler: `get_sessions`; source `backend/api.py:194`.

Direct JSON response keys: `sessions`.

A. POST /api/sessions

Handler: `post_session`; source `backend/api.py:206`.

Direct JSON response keys: `session`.

A. DELETE /api/sessions

Handler: `delete_all_sessions`; source `backend/api.py:211`.

Direct JSON response keys: `deleted_count`, `deleted`, `sessions`, `next_session_id`.

A. GET /api/sessions/{session_id}

Handler: `get_session`; source `backend/api.py:232`.

Direct JSON response keys: `session`, `messages`, `documents`.

A. DELETE /api/sessions/{session_id}

Handler: `delete_session`; source `backend/api.py:242`.

Direct JSON response keys: `deleted`, `sessions`, `next_session_id`.

A. GET /api/sessions/{session_id}/documents

Handler: `get_documents`; source `backend/api.py:264`.

Direct JSON response keys: `documents`.

A. POST /api/sessions/{session_id}/documents/files

Handler: `upload_files`; source `backend/api.py:270`.

Direct JSON response keys: `loaded`, `failed`, `documents`.

Request: multipart field `files`, typed list[UploadFile], required through `File(...)`.

A. POST /api/sessions/{session_id}/documents/urls

Handler: `load_urls`; source `backend/api.py:299`.

Direct JSON response keys: `loaded`, `failed`, `documents`.

A. POST /api/sessions/{session_id}/documents/arxiv

Handler: `load_arxiv_endpoint`; source `backend/api.py:314`.

Direct JSON response keys: `loaded`, `failed`, `documents`.

A. POST /api/sessions/{session_id}/chat

Handler: `chat`; source `backend/api.py:326`.

Response: `StreamingResponse`, media type `text/event-stream`. Stream events and payloads are documented in Chapter 12; do not parse this response as one JSON object.

A. GET /{path:path}

Handler: `frontend`; source `backend/api.py:416`.

Direct JSON response keys: `message`.

B. Complete state and structured-output schemas

Annotations and defaults below are taken directly from class bodies. A missing assigned default generally means a required Pydantic field; TypedDict graph fields describe graph state rather than a request validator. `RerankedDocument` is a frozen dataclass.

B. ChatRequest

Source: `backend/api.py:41`. Bases: `BaseModel`.

- `message`: `str`; no assigned default.

B. UrlLoadRequest

Source: `backend/api.py:45`. Bases: `BaseModel`.

- `urls`: `list[str]`; no assigned default.

B. ArxivLoadRequest

Source: `backend/api.py:49`. Bases: `BaseModel`.

- `query`: `str`; no assigned default.

B. BtwRouteDecision

Source: `backend/models.py:6`. Bases: `BaseModel`.

- `needs_web_search`: `bool`; no assigned default.

B. RouterDecision

Source: `backend/models.py:10`. Bases: `BaseModel`.

- `route`: `Literal['retrieve', 'verify_claim', 'direct_answer']`; no assigned default.

B. RelevancyDecision

Source: `backend/models.py:14`. Bases: `BaseModel`.

- `is_relevant`: `bool`; no assigned default.
- `reason`: `str`; no assigned default.

B. RetrievalSourceDecision

Source: `backend/models.py:19`. Bases: `BaseModel`.

- `source`: `Literal['vectorstore', 'web_search']`; no assigned default.

B. SupersedingPaper

Source: `backend/models.py:23`. Bases: `BaseModel`.

- `title`: `str`; no assigned default.
- `url`: `str`; no assigned default.
- `summary`: `str`; no assigned default.

B. ClaimVerificationResult

Source: `backend/models.py:29`. Bases: `BaseModel`.

- `is_superseded`: `bool`; no assigned default.
- `verdict_summary`: `str`; no assigned default.
- `superseding_papers`: `list[SupersedingPaper]`; no assigned default.

B. RAGState

Source: `backend/rag_graph.py:37`. Bases: `MessagesState`.

- `session_id`: `str`; no assigned default.
- `query`: `str`; no assigned default.
- `route`: `str|None`; no assigned default.
- `retrieval_source`: `str|None`; no assigned default.
- `retrieved_docs`: `list[Document]`; no assigned default.
- `retrieval_attempts`: `int`; no assigned default.
- `claim_verdict`: `str|None`; no assigned default.
- `claim_source`: `str|None`; no assigned default.
- `superseding_papers`: `list[dict]|None`; no assigned default.
- `answer`: `str|None`; no assigned default.
- `is_relevant`: `bool|None`; no assigned default.
- `rewrite_count`: `int`; no assigned default.

B. RetrieverInput

Source: backend/rag_graph.py:128. Bases: BaseModel.

- query: str; default/expression: Field(description='Semanticquerytosearchresearchpaperchunks').
- k: int; default/expression: Field(default=4,ge=1,le=10,description='Numberofchunkstoretrieve').

B. WebSearchInput

Source: backend/rag_graph.py:133. Bases: BaseModel.

- optimized_query: str; default/expression: Field(description='Queryrewrittenandoptimizedforwebsearch').
- max_results: int; default/expression: Field(default=3,ge=1,le=10,description='Numberofwebresultstoreturn').

B. RerankedDocument

Source: backend/reranker.py:26. Bases: none.

- document: Document; no assigned default.
- score: float; no assigned default.
- rerank_rank: int; no assigned default.
- retrieval_candidate_rank: int; no assigned default.

C. Environment-variable reference

This reference reads environment lookups in source, not credential values from .env. Defaults are literal expressions. Provider-selection and precedence rules in Chapter 11 remain essential: an environment model may be bypassed by an explicit factory argument. Restart backend processes after changing import-time settings.

C. GROQ_API_KEY

- backend/providers.py:75 – default: None.
- backend/providers.py:89 – default: None.

C. GROQ_BASE_URL

- backend/providers.py:90 – default: 'https://api.groq.com/openai/v1'.

C. GROQ_CHAT_MODEL

- backend/providers.py:88 – default: None.

C. LLM_PROVIDER

- backend/providers.py:50 – default: ''.

C. OLLAMA_BASE_URL

- backend/providers.py:95 – default: 'http://localhost:11434'.
- backend/providers.py:142 – default: 'http://localhost:11434'.
- evaluate.py:46 – default: 'http://localhost:11434'.

C. OLLAMA_CHAT_MODEL

- backend/providers.py:94 – default: None.
- evaluate.py:45 – default: 'gemma3'.

C. OLLAMA_EMBEDDING_MODEL

- backend/providers.py:141 – default: 'nomic-embed-text'.

C. OLLAMA_EVAL_NUM_CTX

- evaluate.py:48 – default: '4096'.

C. OLLAMA_EVAL_NUM_PREDICT

- evaluate.py:49 – default: '1024'.

C. OLLAMA_GPU_MIN_FREE_MB

- backend/providers.py:29 – default: '5000'.

C. OLLAMA_NUM_CTX

- backend/providers.py:96 – default: '4096'.

C. OLLAMA_NUM_GPU

- backend/providers.py:21 – default: None.

C. OLLAMA_REQUEST_TIMEOUT

- backend/providers.py:98 – default: '300'.
- backend/providers.py:144 – default: '300'.

C. OPENAI_API_KEY

- backend/providers.py:66 – default: None.
- backend/providers.py:75 – default: None.

C. OPENAI_CHAT_MODEL

- backend/providers.py:84 – default: 'gpt-5-mini'.

C. OPENAI_EMBEDDING_MODEL

- backend/providers.py:138 – default: 'text-embedding-3-small'.

C. QDRANT_API_KEY

- backend/vector_store.py:39 – default: None.

C. QDRANT_COLLECTION_PREFIX

- backend/vector_store.py:21 – default: 'scholarlens'.

C. QDRANT_URL

- `backend/vector_store.py:38` – default: `requireddirectlookup`.

C. RERANKER_FINAL_K

- `backend/reranker.py:20` – default: `'6'`.

C. RERANKER_MAX_DOC_CHARS

- `backend/reranker.py:19` – default: `'2500'`.

C. RERANKER_MODEL

- `backend/reranker.py:17` – default: `'BAAI/bge-reranker-base'`.

C. TAVILY_API_KEY

- `backend/btw_handler.py:27` – default: `requireddirectlookup`.
- `backend/rag_graph.py:398` – default: `requireddirectlookup`.
- `backend/rag_graph.py:168` – default: `requireddirectlookup`.
- `backend/rag_graph.py:193` – default: `requireddirectlookup`.
- `smoke_features.py:74` – default: `requireddirectlookup`.

C. Frontend and container-only settings

- `VITE_API_BASE_URL`: empty string by default; used by Vite-built frontend fetches, not a backend runtime setting.
- `SCHOLARLENS_PORT`: Compose default 8020, host publication of Nginx port 80.
- `USER_AGENT`: Compose default ScholarLens/0.2, available to page-loading dependencies.
- `OLLAMA_KEEP_ALIVE`: Compose Ollama setting 10m.
- `OLLAMA_NUM_PARALLEL`: Compose Ollama setting 1.
- `OLLAMA_HOST`: model-pull service endpoint `http://ollama:11434`.
- `NVIDIA_VISIBLE_DEVICES`: `all`; `NVIDIA_DRIVER_CAPABILITIES`: `compute,utility` in Ollama service.
- `PYTHONDONTWRITEBYTECODE=1` and `PYTHONUNBUFFERED=1`: backend image runtime flags.

Compose explicitly sets internal Qdrant/Ollama addresses and an empty Qdrant key. Its full source is included next. The supplied environment file was inspected only for key names; no values are published.

D. Exact build and deployment configuration

These are snapshot copies of the actual configuration, including the empty model-pull command. They are reference material, not corrected deployment files. Chapter 14 explains the necessary operating adjustments.

D. Dockerfile

```
FROM node:20-bookworm-slim AS frontend-build

WORKDIR /app/frontend
COPY frontend/package*.json ./
RUN npm ci
COPY frontend/ ./
RUN npm run build

FROM python:3.12-slim AS runtime

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

RUN apt-get update \
    && apt-get install -y --no-install-recommends build-essential curl \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY backend/ backend/
COPY documents/ documents/
COPY main.py evaluate.py goldens.json sessions.json ./
COPY --from=frontend-build /app/frontend/dist frontend/dist

EXPOSE 8000

CMD ["uvicorn", "backend.api:app", "--host", "0.0.0.0", "--port", "8000"]

FROM nginx:1.27-alpine AS frontend-runtime

COPY frontend/nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=frontend-build /app/frontend/dist /usr/share/nginx/html

EXPOSE 80
```

D. docker-compose.yml

```
name: scholarlens

services:
  ollama:
    image: ollama/ollama:latest
    container_name: scholarlens-ollama
    volumes:
      - ollama_data:/root/.ollama
    gpus: all
    environment:
      NVIDIA_VISIBLE_DEVICES: all
      NVIDIA_DRIVER_CAPABILITIES: compute,utility
      OLLAMA_KEEP_ALIVE: 10m
      OLLAMA_NUM_PARALLEL: 1
    healthcheck:
      test: ["CMD", "ollama", "list"]
      interval: 10s
      timeout: 5s
      retries: 30
    restart: unless-stopped

  ollama-pull:
    image: ollama/ollama:latest
    container_name: scholarlens-ollama-pull
    volumes:
      - ollama_data:/root/.ollama
    environment:
      OLLAMA_HOST: http://ollama:11434
      OLLAMA_CHAT_MODEL: ${OLLAMA_CHAT_MODEL:-llama3.1}
      OLLAMA_EMBEDDING_MODEL: ${OLLAMA_EMBEDDING_MODEL:-nomic-embed-text}
```

```

depends_on:
  ollama:
    condition: service_healthy
entrypoint:
  - /bin/sh
  - -c
  - |
restart: "no"

qdrant:
  image: qdrant/qdrant:latest
  container_name: scholarlens-qdrant
  volumes:
    - qdrant_data:/qdrant/storage
  restart: unless-stopped

backend:
  build:
    context: .
    target: runtime
  image: scholarlens-backend:latest
  container_name: scholarlens-backend
  env_file:
    - .env
  environment:
    LLM_PROVIDER: ${LLM_PROVIDER:-ollama}
    OLLAMA_BASE_URL: http://ollama:11434
    OLLAMA_CHAT_MODEL: ${OLLAMA_CHAT_MODEL:-llama3.1}
    OLLAMA_EMBEDDING_MODEL: ${OLLAMA_EMBEDDING_MODEL:-nomic-embed-text}
    OLLAMA_REQUEST_TIMEOUT: ${OLLAMA_REQUEST_TIMEOUT:-300}
    OLLAMA_NUM_GPU: ${OLLAMA_NUM_GPU:-999}
    OLLAMA_NUM_CTX: ${OLLAMA_NUM_CTX:-4096}
    QDRANT_URL: http://qdrant:6333
    QDRANT_API_KEY: ""
    QDRANT_COLLECTION_PREFIX: ${QDRANT_COLLECTION_PREFIX:-scholarlens}
    USER_AGENT: ${USER_AGENT:-ScholarLens/0.2}
  expose:
    - "8000"
  volumes:
    - ./sessions.json:/app/sessions.json
    - ./checkpoints.db:/app/checkpoints.db
    - ./embedding_cache:/app/embedding_cache
  depends_on:
    qdrant:
      condition: service_started
    ollama-pull:
      condition: service_completed_successfully
  healthcheck:
    test: ["CMD", "curl", "-fsS", "http://127.0.0.1:8000/api/health"]
    interval: 10s
    timeout: 5s
    retries: 30
  restart: unless-stopped

frontend:
  build:
    context: .
    target: frontend-runtime
  image: scholarlens-frontend:latest
  container_name: scholarlens-frontend
  depends_on:
    backend:
      condition: service_healthy
  ports:
    - "${SCHOLARLENS_PORT:-8020}:80"
  healthcheck:
    test: ["CMD-SHELL", "wget -q0- http://127.0.0.1/ >/dev/null"]
    interval: 10s
    timeout: 5s
    retries: 30
  restart: unless-stopped

volumes:
  ollama_data:
  qdrant_data:

```

D. frontend/nginx.conf

```
server {
    listen 80;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    client_max_body_size 100m;

    location /api/ {
        proxy_pass http://scholarlens-backend:8000/api/;
        proxy_http_version 1.1;
        proxy_buffering off;
        proxy_read_timeout 300s;
        proxy_send_timeout 300s;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /assets/ {
        try_files $uri =404;
        access_log off;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    location / {
        try_files $uri /index.html;
    }
}
```

D. frontend/package.json

```
{
  "name": "scholarlens-frontend",
  "private": true,
  "version": "0.2.0",
  "type": "module",
  "scripts": {
    "dev": "vite --host 127.0.0.1",
    "build": "vite build",
    "preview": "vite preview --host 127.0.0.1"
  },
  "dependencies": {
    "@vitejs/plugin-react": "^5.0.0",
    "katex": "^0.16.47",
    "lucide-react": "^0.468.0",
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "react-markdown": "^10.0.0",
    "rehype-katex": "^7.0.1",
    "remark-gfm": "^4.0.0",
    "remark-math": "^6.0.0",
    "typescript": "^5.8.0",
    "vite": "^7.0.0"
  },
  "devDependencies": {
    "playwright": "^1.62.1"
  }
}
```

D. pyproject.toml

```
[project]
name = "scholarlens"
```

```

version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.12"
dependencies = [
    "arxiv>=3.0.0",
    "beautifulsoup4>=4.14.3",
    "chromadb>=1.5.9",
    "deepeval>=4.0.0",
    "fastembed>=0.7.0",
    "fastapi>=0.124.0",
    "langchain-classic>=1.0.7",
    "langchain-community>=0.4.1",
    "langchain-core>=1.3.3",
    "langchain-openai>=1.2.1",
    "langchain-ollama>=0.3.0",
    "langchain-qdrant>=1.1.0",
    "langchain-text-splitters>=1.1.2",
    "langgraph>=1.1.10",
    "langgraph-checkpoint-sqlite>=3.0.3",
    "langsmith>=0.8.3",
    "lxml>=6.1.0",
    "pydantic>=2.13.4",
    "pymupdf>=1.27.2.3",
    "pypdf>=6.10.2",
    "python-dotenv>=1.2.2",
    "python-multipart>=0.0.27",
    "qdrant-client>=1.17.1",
    "sentence-transformers>=5.0.0",
    "tavily-python>=0.7.24",
]

```

D. .python-version

```
3.12
```

D. .dockerignore

```

.env
.venv/
__pycache__/
*.py[cod]
*.pyo
embedding_cache/
*.db
*.db-shm
*.db-wal
.deepeval/
.claude/
tests/
graph_diag.ipynb
eval_results.json
report.md
CLAUDE.md
.git/
.ruff_cache/
.mypy_cache/
.pytest_cache/
graph.png
frontend/node_modules/
frontend/dist/

```

D. Ignore-file implications

The Git ignore file excludes environments, caches, root SQLite files, tests/, graph_diag.ipynb, some local reports, and other development outputs. The Docker ignore file excludes frontend dependencies/build output, caches,

database files, and tests. The source build recreates frontend/dist. Neither file is an access-control mechanism. A tests/ ignore rule can hide newly added regression tests from ordinary Git status until intentionally adjusted.

E. Complete Python function and class index

All definitions in the active backend, root scripts, and archived backend files are indexed. Nested helper functions are included. Line ranges refer to this snapshot. Archived code is not selected by the runtime entrypoint.

E. backend/___init___py

Package marker only; no function or class definitions.

E. backend/api.py

- `ChatRequest` – class, lines 41-42.
- `UrlLoadRequest` – class, lines 45-46.
- `ArxivLoadRequest` – class, lines 49-50.
- `load_sessions` – function, lines 53-57.
- `save_sessions` – function, lines 60-61.
- `create_session` – function, lines 64-75.
- `ensure_session` – function, lines 78-82.
- `maybe_rename_session` – function, lines 85-109.
- `serialize_state` – function, lines 112-134.
- `serialize_retrieved_context` – function, lines 137-146.
- `load_session_chats` – function, lines 149-181.
- `sse` – function, lines 184-185.
- `health` – function, lines 189-190.
- `get_sessions` – function, lines 194-202.
- `post_session` – function, lines 206-207.
- `delete_all_sessions` – function, lines 211-228.
- `get_session` – function, lines 232-238.
- `delete_session` – function, lines 242-260.
- `get_documents` – function, lines 264-266.
- `upload_files` – function, lines 270-295.
- `load_urls` – function, lines 299-310.
- `load_arxiv_endpoint` – function, lines 314-322.
- `chat` – function, lines 326-408.
- `chat.event_stream` – function, lines 332-406.
- `frontend` – function, lines 416-420.

E. backend/btw_handler.py

- `handle_btw` – function, lines 16-46. Off-topic side channel – never touches the vector store or checkpoint.

E. backend/models.py

- `BtwRouteDecision` – class, lines 6-7.
- `RouterDecision` – class, lines 10-11.
- `RelevancyDecision` – class, lines 14-16.
- `RetrievalSourceDecision` – class, lines 19-20.
- `SupersedingPaper` – class, lines 23-26.
- `ClaimVerificationResult` – class, lines 29-32.

E. backend/paper_loader.py

- `_stamp_title` – function, lines 26-29.
- `load_pdf` – function, lines 32-34.
- `load_text` – function, lines 37-39.
- `load_markdown` – function, lines 42-44.
- `load_webpage` – function, lines 47-50.
- `_extract_arxiv_id` – function, lines 53-58. Return bare ArXiv ID (no version suffix) if one appears in the query.
- `_arxiv_api_metadata` – function, lines 61-76. Fetch paper title and authors by ID from the ArXiv Atom API.
- `_arxiv_search` – function, lines 79-89. Search ArXiv Atom API by title phrase and return the top result's bare paper ID.
- `_download_arxiv_pdf` – function, lines 92-110. Download an ArXiv PDF, trying the primary host and a fallback host.
- `_load_arxiv_by_id` – function, lines 113-149. Download and chunk an ArXiv paper PDF by its bare ID.
- `load_arxiv` – function, lines 152-154.
- `load_document` – function, lines 157-168. Dispatch to the appropriate loader based on URL prefix or file extension.

E. backend/providers.py

- `_auto_ollama_num_gpu` – function, lines 15-46. Return Ollama's `num_gpu` setting.
- `_provider` – function, lines 49-50.
- `is_ollama` – function, lines 53-55. Return whether the active LLM provider is local Ollama.
- `_use_openai` – function, lines 58-66.
- `_use_groq` – function, lines 69-75.
- `get_chat_model` – function, lines 78-104. Return the configured chat model.
- `get_structured_output` – function, lines 107-129. Use native structured output, with JSON parsing for Ollama.
- `get_embeddings` – function, lines 132-150. Return the configured embedding model.

E. backend/rag_graph.py

- `RAGState` – class, lines 37-49.
- `_format_recent_messages` – function, lines 52-65.
- `router_node` – function, lines 109-123.
- `RetrieverInput` – class, lines 128-130.
- `WebSearchInput` – class, lines 133-135.
- `retrieve_from_vectorstore` – function, lines 141-156. Search the uploaded research paper vector store for relevant passages.
- `web_search` – function, lines 160-187. Search the web for current or supplementary information using Tavily.
- `_search_web_documents` – function, lines 190-206. Retrieve web results for provider modes that do not support tool calls.
- `agent_node` – function, lines 284-334.
- `relevancy_check_node` – function, lines 337-352.
- `query_rewrite_node` – function, lines 355-377.
- `verify_claim_node` – function, lines 396-445.
- `generate_answer_node` – function, lines 448-550.
- `route_query` – function, lines 558-559.
- `agent_routing` – function, lines 562-571.
- `after_relevancy_routing` – function, lines 574-579.
- `build_graph` – function, lines 582-628.

E. backend/reranker.py

- `_normalize_model_name` – function, lines 7-13.
- `RerankedDocument` – class, lines 26-30.
- `_get_cross_encoder` – function, lines 33-39.
- `rerank_documents` – function, lines 42-88.

E. backend/vector_store.py

- `get_embedding_dimension` – function, lines 46-51. Read the configured embedding model's dimension once for Qdrant.
- `_collection_has_hybrid_schema` – function, lines 54-63.
- `get_collection_name` – function, lines 69-70.
- `get_vectorstore` – function, lines 73-105.
- `add_paper` – function, lines 110-111.
- `delete_session_collection` – function, lines 114-124.
- `list_papers` – function, lines 127-156.
- `search` – function, lines 159-169.

E. backend copy/__init__.py

Package marker only; no function or class definitions.

E. backend copy/api.py

- `ChatRequest` – class, lines 41-42.
- `UrlLoadRequest` – class, lines 45-46.
- `ArxivLoadRequest` – class, lines 49-50.
- `load_sessions` – function, lines 53-57.
- `save_sessions` – function, lines 60-61.
- `create_session` – function, lines 64-75.
- `ensure_session` – function, lines 78-82.
- `maybe_rename_session` – function, lines 85-109.
- `serialize_state` – function, lines 112-134.
- `serialize_retrieved_context` – function, lines 137-146.
- `load_session_chats` – function, lines 149-181.
- `sse` – function, lines 184-185.
- `health` – function, lines 189-190.
- `get_sessions` – function, lines 194-202.
- `post_session` – function, lines 206-207.
- `delete_all_sessions` – function, lines 211-228.
- `get_session` – function, lines 232-238.
- `delete_session` – function, lines 242-260.
- `get_documents` – function, lines 264-266.
- `upload_files` – function, lines 270-295.
- `load_urls` – function, lines 299-310.
- `load_arxiv_endpoint` – function, lines 314-322.
- `chat` – function, lines 326-408.
- `chat.event_stream` – function, lines 332-406.
- `frontend` – function, lines 416-420.

E. backend copy/btw_handler.py

- `handle_btw` – function, lines 16-46. Off-topic side channel – never touches the vector store or checkpoint.

E. backend copy/models.py

- `BtwRouteDecision` – class, lines 6-7.
- `RouterDecision` – class, lines 10-11.
- `RelevancyDecision` – class, lines 14-16.
- `RetrievalSourceDecision` – class, lines 19-20.
- `SupersedingPaper` – class, lines 23-26.
- `ClaimVerificationResult` – class, lines 29-32.

E. backend copy/paper_loader.py

- `_stamp_title` – function, lines 26-29.
- `load_pdf` – function, lines 32-34.
- `load_text` – function, lines 37-39.
- `load_markdown` – function, lines 42-44.
- `load_webpage` – function, lines 47-50.
- `_extract_arxiv_id` – function, lines 53-58. Return bare ArXiv ID (no version suffix) if one appears in the query.
- `_arxiv_api_metadata` – function, lines 61-76. Fetch paper title and authors by ID from the ArXiv Atom API.
- `_arxiv_search` – function, lines 79-89. Search ArXiv Atom API by title phrase and return the top result's bare paper ID.
- `_download_arxiv_pdf` – function, lines 92-110. Download an ArXiv PDF, trying the primary host and a fallback host.
- `_load_arxiv_by_id` – function, lines 113-149. Download and chunk an ArXiv paper PDF by its bare ID.
- `load_arxiv` – function, lines 152-154.
- `load_document` – function, lines 157-168. Dispatch to the appropriate loader based on URL prefix or file extension.

E. backend copy/providers.py

- `_auto_ollama_num_gpu` – function, lines 15-46. Return Ollama's `num_gpu` setting.
- `_provider` – function, lines 49-50.
- `is_ollama` – function, lines 53-55. Return whether the active LLM provider is local Ollama.
- `_use_openai` – function, lines 58-66.
- `_use_groq` – function, lines 69-75.
- `get_chat_model` – function, lines 78-104. Return the configured chat model.
- `get_structured_output` – function, lines 107-129. Use native structured output, with JSON parsing for Ollama.
- `get_embeddings` – function, lines 132-150. Return the configured embedding model.

E. backend copy/rag_graph.py

- `RAGState` – class, lines 36-48.
- `router_node` – function, lines 90-102.
- `RetrieverInput` – class, lines 107-109.
- `WebSearchInput` – class, lines 112-114.
- `retrieve_from_vectorstore` – function, lines 120-135. Search the uploaded research paper vector store for relevant passages.
- `web_search` – function, lines 139-166. Search the web for current or supplementary information using Tavily.
- `_search_web_documents` – function, lines 169-185. Retrieve web results for provider modes that do not support tool calls.
- `agent_node` – function, lines 262-295.
- `relevancy_check_node` – function, lines 298-312.

- `query_rewrite_node` – function, lines 315-331.
- `verify_claim_node` – function, lines 350-399.
- `generate_answer_node` – function, lines 402-473.
- `route_query` – function, lines 481-482.
- `agent_routing` – function, lines 485-494.
- `after_relevancy_routing` – function, lines 497-502.
- `build_graph` – function, lines 505-551.

E. backend copy/vector_store.py

- `get_embedding_dimension` – function, lines 40-45. Read the configured embedding model's dimension once for Qdrant.
- `_collection_dimension` – function, lines 48-52.
- `get_collection_name` – function, lines 60-61.
- `get_vectorstore` – function, lines 64-91.
- `add_paper` – function, lines 96-97.
- `delete_session_collection` – function, lines 100-110.
- `list_papers` – function, lines 113-142.
- `search` – function, lines 145-155.

E. backend streamlit/___init___py

Package marker only; no function or class definitions.

E. backend streamlit/api.py

- `ChatRequest` – class, lines 41-42.
- `UrlLoadRequest` – class, lines 45-46.
- `ArxivLoadRequest` – class, lines 49-50.
- `load_sessions` – function, lines 53-57.
- `save_sessions` – function, lines 60-61.
- `create_session` – function, lines 64-75.
- `ensure_session` – function, lines 78-82.
- `maybe_rename_session` – function, lines 85-109.
- `serialize_state` – function, lines 112-134.
- `serialize_retrieved_context` – function, lines 137-146.
- `load_session_chats` – function, lines 149-181.
- `sse` – function, lines 184-185.
- `health` – function, lines 189-190.
- `get_sessions` – function, lines 194-202.
- `post_session` – function, lines 206-207.
- `delete_all_sessions` – function, lines 211-228.
- `get_session` – function, lines 232-238.

- `delete_session` – function, lines 242-260.
- `get_documents` – function, lines 264-266.
- `upload_files` – function, lines 270-295.
- `load_urls` – function, lines 299-310.
- `load_arxiv_endpoint` – function, lines 314-322.
- `chat` – function, lines 326-408.
- `chat.event_stream` – function, lines 332-406.
- `frontend` – function, lines 416-420.

E. backend streamlit/btw_handler.py

- `handle_btw` – function, lines 16-46. Off-topic side channel – never touches the vector store or checkpoint.

E. backend streamlit/models.py

- `BtwRouteDecision` – class, lines 6-7.
- `RouterDecision` – class, lines 10-11.
- `RelevancyDecision` – class, lines 14-16.
- `RetrievalSourceDecision` – class, lines 19-20.
- `SupersedingPaper` – class, lines 23-26.
- `ClaimVerificationResult` – class, lines 29-32.

E. backend streamlit/paper_loader.py

- `_stamp_title` – function, lines 26-29.
- `load_pdf` – function, lines 32-34.
- `load_text` – function, lines 37-39.
- `load_markdown` – function, lines 42-44.
- `load_webpage` – function, lines 47-50.
- `_extract_arxiv_id` – function, lines 53-58. Return bare ArXiv ID (no version suffix) if one appears in the query.
- `_arxiv_api_metadata` – function, lines 61-76. Fetch paper title and authors by ID from the ArXiv Atom API.
- `_arxiv_search` – function, lines 79-89. Search ArXiv Atom API by title phrase and return the top result's bare paper ID.
- `_download_arxiv_pdf` – function, lines 92-110. Download an ArXiv PDF, trying the primary host and a fallback host.
- `_load_arxiv_by_id` – function, lines 113-149. Download and chunk an ArXiv paper PDF by its bare ID.
- `load_arxiv` – function, lines 152-154.
- `load_document` – function, lines 157-168. Dispatch to the appropriate loader based on URL prefix or file extension.

E. backend streamlit/providers.py

- `_auto_ollama_num_gpu` – function, lines 15-46. Return Ollama’s `num_gpu` setting.
- `_provider` – function, lines 49-50.
- `is_ollama` – function, lines 53-55. Return whether the active LLM provider is local Ollama.
- `_use_openai` – function, lines 58-66.
- `_use_groq` – function, lines 69-75.
- `get_chat_model` – function, lines 78-104. Return the configured chat model.
- `get_structured_output` – function, lines 107-129. Use native structured output, with JSON parsing for Ollama.
- `get_embeddings` – function, lines 132-150. Return the configured embedding model.

E. backend streamlit/rag_graph.py

- `RAGState` – class, lines 36-48.
- `router_node` – function, lines 90-102.
- `RetrieverInput` – class, lines 107-109.
- `WebSearchInput` – class, lines 112-114.
- `retrieve_from_vectorstore` – function, lines 120-135. Search the uploaded research paper vector store for relevant passages.
- `web_search` – function, lines 139-166. Search the web for current or supplementary information using Tavily.
- `_search_web_documents` – function, lines 169-185. Retrieve web results for provider modes that do not support tool calls.
- `agent_node` – function, lines 262-295.
- `relevancy_check_node` – function, lines 298-312.
- `query_rewrite_node` – function, lines 315-331.
- `verify_claim_node` – function, lines 350-399.
- `generate_answer_node` – function, lines 402-473.
- `route_query` – function, lines 481-482.
- `agent_routing` – function, lines 485-494.
- `after_relevancy_routing` – function, lines 497-502.
- `build_graph` – function, lines 505-551.

E. backend streamlit/vector_store.py

- `get_embedding_dimension` – function, lines 40-45. Read the configured embedding model’s dimension once for Qdrant.
- `_collection_dimension` – function, lines 48-52.
- `get_collection_name` – function, lines 60-61.
- `get_vectorstore` – function, lines 64-91.
- `add_paper` – function, lines 96-97.
- `delete_session_collection` – function, lines 100-110.
- `list_papers` – function, lines 113-142.
- `search` – function, lines 145-155.

E. evaluate.py

- `get_evaluation_model` – function, lines 41-52. Use the same local Ollama configuration as the application when available.
- `generate_goldens` – function, lines 55-71.
- `load_goldens` – function, lines 74-75.
- `run_rag_query` – function, lines 78-93.
- `main` – function, lines 96-170.

E. main.py

- `main` – function, lines 1-2.

E. smoke_features.py

- `record` – function, lines 28-31.
- `run` – function, lines 34-314.

F. Frontend function and style reference

Named function declarations in the complete JSX module are listed below, including nested event handlers. Inline arrow callbacks are part of those parent components and are not separate public interfaces.

- `cls` – frontend/src/main.jsx:40.
- `friendlyError` – frontend/src/main.jsx:44.
- `api` – frontend/src/main.jsx:57.
- `copyTextToClipboard` – frontend/src/main.jsx:72.
- `formatDate` – frontend/src/main.jsx:89.
- `parseSseBlock` – frontend/src/main.jsx:102.
- `App` – frontend/src/main.jsx:113.
- `refreshSessions` – frontend/src/main.jsx:173.
- `loadSession` – frontend/src/main.jsx:183.
- `createSession` – frontend/src/main.jsx:190.
- `deleteSession` – frontend/src/main.jsx:200.
- `deleteAllSessions` – frontend/src/main.jsx:232.
- `reloadDocuments` – frontend/src/main.jsx:256.
- `handleUpload` – frontend/src/main.jsx:262.
- `loadUrls` – frontend/src/main.jsx:280.
- `loadArxiv` – frontend/src/main.jsx:297.
- `sendPrompt` – frontend/src/main.jsx:313.
- `patchAssistant` – frontend/src/main.jsx:404.
- `sendMessage` – frontend/src/main.jsx:410.
- `stopGeneration` – frontend/src/main.jsx:415.
- `startSourcePanelResize` – frontend/src/main.jsx:419.

- `retryLastPrompt` – `frontend/src/main.jsx:452`.
- `openTurnDetails` – `frontend/src/main.jsx:456`.
- `Sidebar` – `frontend/src/main.jsx:542`.
- `requestDelete` – `frontend/src/main.jsx:557`.
- `cancelDelete` – `frontend/src/main.jsx:562`.
- `confirmDelete` – `frontend/src/main.jsx:567`.
- `confirmDeleteAllChats` – `frontend/src/main.jsx:573`.
- `TopBar` – `frontend/src/main.jsx:698`.
- `ConversationViewport` – `frontend/src/main.jsx:732`.
- `EmptyState` – `frontend/src/main.jsx:757`.
- `ChatMessage` – `frontend/src/main.jsx:784`.
- `Composer` – `frontend/src/main.jsx:827`.
- `SourcePanel` – `frontend/src/main.jsx:875`.
- `Notice` – `frontend/src/main.jsx:1013`.
- `RetrievedContext` – `frontend/src/main.jsx:1025`.
- `GraphState` – `frontend/src/main.jsx:1054`.
- `handleCopy` – `frontend/src/main.jsx:1076`.

F. CSS custom properties

Property	Initial value
<code>--bg</code>	<code>#0f1115</code>
<code>--sidebar</code>	<code>#17191f</code>
<code>--surface</code>	<code>#1b1e25</code>
<code>--surface-2</code>	<code>#222630</code>
<code>--surface-3</code>	<code>#2a2f3a</code>
<code>--line</code>	<code>#303541</code>
<code>--line-soft</code>	<code>rgba(255, 255, 255, 0.08)</code>
<code>--text</code>	<code>#f3f4f6</code>
<code>--muted</code>	<code>#a7adb8</code>
<code>--muted-2</code>	<code>#7f8794</code>
<code>--accent</code>	<code>#10a37f</code>
<code>--accent-hover</code>	<code>#0d8f70</code>
<code>--danger</code>	<code>#ef4444</code>
<code>--warning</code>	<code>#f59e0b</code>
<code>--radius</code>	<code>8px</code>
<code>--sidebar-width</code>	<code>276px</code>
<code>--source-panel-width</code>	<code>360px</code>
<code>--topbar-height</code>	<code>58px</code>

F. Responsive breakpoints

- `@media(min-width:1200px)` – `frontend/src/styles.css:1202`.
- `@media(max-width:1199px)` – `frontend/src/styles.css:1208`.
- `@media(max-width:1024px)` – `frontend/src/styles.css:1219`.

- `@media(max-width:768px) – frontend/src/styles.css:1233.`
- `@media(max-width:390px) – frontend/src/styles.css:1335.`

The complete CSS contains shell/sidebar navigation, chat/Markdown/math presentation, composer sizing, source ingestion cards, inspection panels, resize handles, notifications, animation, and responsive overrides. Chapter 15 explains their behavior.

G. Dependency and artifact provenance

G. Python lockfile inventory

The uv lockfile records 193 package entries; Python constraint `>=3.12`. These are resolved records, not the reviewer interpreter's installed packages. Docker pip installation uses a different input, `requirements.txt`, whose full contents follow.

Package	Version	Package	Version
aiohappyeyeballs	2.6.1	nvidia-nccl-cu13	2.29.7
aiohttp	3.13.5	nvidia-nvjitlink	13.3.33
aiosignal	1.4.0	nvidia-nvshmem-cu13	3.4.5
aiosqlite	0.22.1	nvidia-nvtx	13.0.85
annotated-doc	0.0.4	oauthlib	3.3.1
annotated-types	0.7.0	ollama	0.6.2
anyio	4.13.0	onnxruntime	1.26.0
arxiv	3.0.0	openai	2.36.0
attrs	26.1.0	opentelemetry-api	1.41.1
backoff	2.2.1	opentelemetry-exporter-otlp-proto-common	1.41.1
bcrypt	5.0.0	opentelemetry-exporter-otlp-proto-grpc	1.41.1
beautifulsoup4	4.14.3	opentelemetry-proto	1.41.1
build	1.5.0	opentelemetry-sdk	1.41.1
certifi	2026.4.22	opentelemetry-semantic-conventions	0.62b1
charset-normalizer	3.4.7	orjson	3.11.9
chromadb	1.5.9	ormsgpack	1.12.2
click	8.3.3	overrides	7.7.0
colorama	0.4.6	packaging	26.2
cuda-bindings	13.3.1	pillow	12.3.0
cuda-pathfinder	1.6.0	pluggy	1.6.0
cuda-toolkit	13.0.3.0	portalocker	3.2.0
dataclasses-json	0.6.7	posthog	7.14.0
deepeval	4.0.0	propcache	0.4.1
distro	1.9.0	protobuf	6.33.6
durationpy	0.10	py-rust-stemmers	0.1.8
execnet	2.1.2	pybase64	1.4.3
fastapi	0.141.1	pydantic	2.13.4
fastembed	0.8.0	pydantic-core	2.46.4
feedparser	6.0.12	pydantic-settings	2.14.0
filelock	3.29.0	pyfiglet	1.0.4
flatbuffers	25.12.19	pygments	2.20.0
frozenset	1.8.0	pymupdf	1.27.2.3
fsspec	2026.4.0	pypdf	6.10.2
googleapis-common-protos	1.75.0	pypika	0.51.1
greenlet	3.5.0	pyproject-hooks	1.2.0
grpcio	1.80.0	pytest	9.0.3

Package	Version	Package	Version
h11	0.16.0	pytest-asyncio	1.3.0
h2	4.3.0	pytest-repeat	0.9.4
hf-xet	1.5.0	pytest-rerunfailures	16.1
hpack	4.1.0	pytest-xdist	3.8.0
httpcore	1.0.9	python-dateutil	2.9.0.post0
httptools	0.7.1	python-dotenv	1.2.2
httpx	0.28.1	python-multipart	0.0.32
httpx-sse	0.4.3	pywin32	311
huggingface-hub	1.14.0	pyyaml	6.0.3
hyperframe	6.1.0	qdrant-client	1.17.1
idna	3.13	referencing	0.37.0
importlib-metadata	8.7.1	regex	2026.4.4
importlib-resources	7.1.0	requests	2.33.1
iniconfig	2.3.0	requests-oauthlib	2.0.0
jinja2	3.1.6	requests-toolbelt	1.0.0
jiter	0.14.0	rich	14.3.4
joblib	1.5.3	rpds-py	0.30.0
jsonpatch	1.33	safetensors	0.8.0
jsonpointer	3.1.1	scholarlens	0.1.0
jsonschema	4.26.0	scikit-learn	1.9.0
jsonschema-specifications	2025.9.1	scipy	1.18.0
kubernetes	35.0.0	sentence-transformers	5.6.1
langchain-classic	1.0.7	sentry-sdk	2.59.0
langchain-community	0.4.1	setuptools	82.0.1
langchain-core	1.3.3	sgmllib3k	1.0.0
langchain-ollama	1.1.0	shellingham	1.5.4
langchain-openai	1.2.1	six	1.17.0
langchain-protocol	0.0.15	sniffio	1.3.1
langchain-qdrant	1.1.0	soupsieve	2.8.3
langchain-text-splitters	1.1.2	sqlalchemy	2.0.49
langgraph	1.1.10	sqlite-vec	0.1.9
langgraph-checkpoint	4.0.3	starlette	1.0.0
langgraph-checkpoint-sqlite	3.0.3	sympy	1.14.0
langgraph-prebuilt	1.0.13	tabulate	0.9.0
langgraph-sdk	0.3.14	tavily-python	0.7.24
langsmith	0.8.3	tenacity	9.1.4
loguru	0.7.3	threadpoolctl	3.6.0
lxml	6.1.0	tiktoken	0.12.0
markdown-it-py	4.2.0	tokenizers	0.22.2
markupsafe	3.0.3	torch	2.13.0
marshmallow	3.26.2	tqdm	4.67.3
mdurl	0.1.2	transformers	5.14.1
mmh3	5.2.1	triton	3.7.1
mpmath	1.3.0	typer	0.25.1
multidict	6.7.1	typing-extensions	4.15.0
mypy-extensions	1.1.0	typing-inspect	0.9.0
narwhals	2.24.0	typing-inspection	0.4.2
nest-asyncio	1.6.0	urllib3	2.7.0
networkx	3.6.1	uuid-utils	0.14.1
numpy	2.4.4	uvicorn	0.46.0
nvidia-cublas	13.1.1.3	uvloop	0.22.1
nvidia-cuda-cupti	13.0.85	watchfiles	1.1.1
nvidia-cuda-nvrtc	13.0.88	websocket-client	1.9.0

Package	Version	Package	Version
nvidia-cuda-runtime	13.0.96	websockets	16.0
nvidia-cudnn-cu13	9.20.0.48	wheel	0.47.0
nvidia-cufft	12.0.0.61	win32-setctime	1.2.0
nvidia-cufile	1.15.1.6	xxhash	3.7.0
nvidia-curand	10.4.0.35	yaml	1.23.0
nvidia-cusolver	12.0.4.66	zipp	3.23.1
nvidia-cusparse	12.6.3.3	zstandard	0.25.0
nvidia-cusparselt-cu13	0.8.1		

G. Docker Python requirements

```

aiohappyeyeballs==2.6.1
aiohttp==3.13.5
aiosignal==1.4.0
aiosqlite==0.22.1
altair==6.1.0
annotated-doc==0.0.4
annotated-types==0.7.0
anyio==4.13.0
arxiv==3.0.0
asttokens==3.0.1
attrs==26.1.0
backoff==2.2.1
bcrypt==5.0.0
beautifulsoup4==4.14.3
blinker==1.9.0
build==1.5.0
cachetools==7.1.1
certifi==2026.4.22
charset-normalizer==3.4.7
chromadb==1.5.9
click==8.3.3
colorama==0.4.6
comm==0.2.3
dataclasses-json==0.6.7
debugpy==1.8.20
decorator==5.2.1
deepeval==4.0.0
distro==1.9.0
durationpy==0.10
execnet==2.1.2
executing==2.2.1
fastembed>=0.7.0
feedparser==6.0.12
filelock==3.29.0
fastapi==0.141.1
flatbuffers==25.12.19
frozenlist==1.8.0
fsspec==2026.4.0
gitdb==4.0.12
gitpython==3.1.50
googleapis-common-protos==1.75.0
greenlet==3.5.0
grpcio==1.80.0
h11==0.16.0
h2==4.3.0
hf-xet==1.5.0
hpack==4.1.0
httpcore==1.0.9
httpx==0.7.1
httpx==0.28.1
httpx-sse==0.4.3
huggingface-hub==1.14.0
hyperframe==6.1.0
idna==3.13
importlib-metadata==8.7.1
importlib-resources==7.1.0
iniconfig==2.3.0

```

```
ipykernel==7.2.0
ipython==9.13.0
ipython-pygments-lexers==1.1.1
itsdangerous==2.2.0
jedi==0.20.0
jinja2==3.1.6
jiter==0.14.0
jsonpatch==1.33
jsonpointer==3.1.1
jsonschema==4.26.0
jsonschema-specifications==2025.9.1
jupyter-client==8.8.0
jupyter-core==5.9.1
kubernetes==35.0.0
langchain-classic==1.0.7
langchain-community==0.4.1
langchain-core==1.3.3
langchain-ollama==1.1.0
langchain-openai==1.2.1
langchain-protocol==0.0.15
langchain-qdrant==1.1.0
langchain-text-splitters==1.1.2
langgraph==1.1.10
langgraph-checkpoint==4.0.3
langgraph-checkpoint-sqlite==3.0.3
langgraph-prebuilt==1.0.13
langgraph-sdk==0.3.14
langsmith==0.8.3
lxml==6.1.0
markdown-it-py==4.2.0
markupsafe==3.0.3
marshmallow==3.26.2
matplotlib-inline==0.2.1
mdurl==0.1.2
mmh3==5.2.1
multidict==6.7.1
mypy-extensions==1.1.0
narwhals==2.20.0
nest-asyncio==1.6.0
numpy==2.4.4
oauthlib==3.3.1
onnxruntime==1.26.0
openai==2.36.0
opentelemetry-api==1.41.1
opentelemetry-exporter-otlp-proto-common==1.41.1
opentelemetry-exporter-otlp-proto-grpc==1.41.1
opentelemetry-proto==1.41.1
opentelemetry-sdk==1.41.1
opentelemetry-semantic-conventions==0.62b1
orjson==3.11.9
ormsgpack==1.12.2
overrides==7.7.0
packaging==26.2
pandas==3.0.2
parso==0.8.7
pillow==12.2.0
platformdirs==4.9.6
pluggy==1.6.0
portalocker==3.2.0
posthog==7.14.0
prompt-toolkit==3.0.52
procache==0.4.1
protobuf==6.33.6
psutil==7.2.2
pure-eval==0.2.3
pyarrow==24.0.0
pybase64==1.4.3
pydantic==2.13.4
pydantic-core==2.46.4
pydantic-settings==2.14.0
pydeck==0.9.2
pyfiglet==1.0.4
pygments==2.20.0
pymupdf==1.27.2.3
pypdf==6.10.2
pypika==0.51.1
```

```

pyproject-hooks==1.2.0
pytest==9.0.3
pytest-asyncio==1.3.0
pytest-repeat==0.9.4
pytest-rerunfailures==16.1
pytest-xdist==3.8.0
python-dateutil==2.9.0.post0
python-dotenv==1.2.2
python-multipart==0.0.32
pyyaml==6.0.3
pyzmq==27.1.0
qdrant-client==1.17.1
referencing==0.37.0
regex==2026.4.4
requests==2.33.1
requests-oauthlib==2.0.0
requests-toolbelt==1.0.0
rich==14.3.4
rpds-py==0.30.0
sentry-sdk==2.59.0
sentence-transformers>=5.0.0
setuptools==82.0.1
sgmllib3k==1.0.0
shellingham==1.5.4
six==1.17.0
smmap==5.0.3
sniffio==1.3.1
soupsieve==2.8.3
sqlalchemy==2.0.49
sqlite-vec==0.1.9
stack-data==0.6.3
starlette==1.0.0
tabulate==0.9.0
tavily-python==0.7.24
tenacity==9.1.4
tiktoken==0.12.0
tokenizers==0.22.2
toml==0.10.2
tornado==6.5.5
tqdm==4.67.3
traitlets==5.15.0
typer==0.25.1
typing-extensions==4.15.0
typing-inspect==0.9.0
typing-inspection==0.4.2
tzdata==2026.2
urllib3==2.7.0
uuid-utils==0.14.1
uvicorn==0.46.0
watchdog==6.0.0
watchfiles==1.1.1
wcwidth==0.7.0
websocket-client==1.9.0
websockets==16.0
wheel==0.47.0
xxhash==3.7.0
yarl==1.23.0
zipp==3.23.1
zstandard==0.25.0

```

G. Frontend direct dependency resolutions

The npm lockfile uses lockfileVersion 3 and records 241 package paths including the root. Direct requested and resolved versions are listed here. The complete transitive name/version/path inventory is also emitted as frontend_dependency_inventory.json beside this PDF.

Dependency	Requested	Resolved
@vitejs/plugin-react	~5.0.0	5.2.0
katex	~0.16.47	0.16.47
lucide-react	~0.468.0	0.468.0

Dependency	Requested	Resolved
react	~19.0.0	19.2.8
react-dom	~19.0.0	19.2.8
react-markdown	~10.0.0	10.1.0
rehype-katex	~7.0.1	7.0.1
remark-gfm	~4.0.0	4.0.1
remark-math	~6.0.0	6.0.0
typescript	~5.8.0	5.9.3
vite	~7.0.0	7.3.6
playwright	~1.62.1	1.62.1

G. Existing data and runtime artifacts

- Sample document: 34-page PDF under documents/, used by evaluate.py; report contents were not externally fact-checked.
- goldens.json: ten input/expected_output records; old result file: ten evaluated records, five overall successes.
- sessions.json: one metadata record at review time; no session labels or conversations reproduced here.
- Application/evaluation SQLite schema: checkpoints and writes tables; journal sidecars are present.
- Embedding cache: 2,302 files observed; contents not copied into this manual.
- .deepeval cache files hold test lookup/run metadata; telemetry contents are not published.
- graph.png: supplied static topology illustration; the active direct-retrieval distinction is explained in Chapter 9.
- .env exists and is excluded from both published content and source fingerprints. Runtime records and generated third-party artifacts are likewise excluded from the source table.

G. Archived source comparison

File	Archive copies match each other	Active vs archive
__init__.py	yes	identical
api.py	yes	identical
btw_handler.py	yes	identical
models.py	yes	identical
paper_loader.py	yes	identical
providers.py	yes	identical
rag_graph.py	yes	different
vector_store.py	yes	different

backend/reranker.py exists only in the active backend. Bytecode differences are not source differences.

H. Review coverage, validation, and source fingerprints

The reproducible inventory contains 52 original source/configuration/fixture files, including 28 Python files. Full SHA-256 hashes and byte sizes are in source_inventory.json. File exclusions are explicit in Chapter 1. This table is not a hash inventory of private runtime data.

Original file	Bytes	SHA-256 prefix
.dockerignore	258	482ced197b6cf051
.gitignore	4815	84476b5a61fc6f26
.python-version	5	7b55f8e67b5623c4
DOCKER_GUIDE.md	1088	a4c7df6381f54d90
Dockerfile	890	779c01c5ef681d56
README.md	3624	a9200e212681b785
backend/__init__.py	1	01ba4719c80b6fe9
backend/api.py	13779	1dfb04767afcd0f4
backend/btw_handler.py	1762	a56883cc240e0f07
backend/models.py	613	348cfa1ffe491343
backend/paper_loader.py	6223	0f2e12a5583186ff
backend/providers.py	4871	136ca618672b8be9
backend/rag_graph.py	27286	d1c8c3b845abd577
backend/reranker.py	2414	dfee60264a1aa0a7
backend/vector_store.py	6134	a41a12561b46a792
backendcopy/__init__.py	1	01ba4719c80b6fe9
backendcopy/api.py	13779	1dfb04767afcd0f4
backendcopy/btw_handler.py	1762	a56883cc240e0f07
backendcopy/models.py	613	348cfa1ffe491343
backendcopy/paper_loader.py	6223	0f2e12a5583186ff
backendcopy/providers.py	4871	136ca618672b8be9
backendcopy/rag_graph.py	23241	7d14b4b565f6b7ee
backendcopy/vector_store.py	5692	0607fddd57576818
backendstreamlit/__init__.py	1	01ba4719c80b6fe9
backendstreamlit/api.py	13779	1dfb04767afcd0f4
backendstreamlit/btw_handler.py	1762	a56883cc240e0f07
backendstreamlit/models.py	613	348cfa1ffe491343
backendstreamlit/paper_loader.py	6223	0f2e12a5583186ff
backendstreamlit/providers.py	4871	136ca618672b8be9
backendstreamlit/rag_graph.py	23241	7d14b4b565f6b7ee
backendstreamlit/vector_store.py	5692	0607fddd57576818
docker-compose.yml	2785	8bcd13faef44fc4f
docker_install_on_AWS.txt	909	01715a69b08b3b2e
documents/Openclaw_Research_Report.pdf	940600	3bc9c0fe983c58bb
eval_results_old.json	45628	4c9049a9abc4d504
evaluate.py	5967	c92672ac2274e4ac
frontend/index.html	363	157daec9852c39e4
frontend/nginx.conf	769	b1dd211f8e91386b
frontend/package-lock.json	129433	64519ce1a19ef814
frontend/package.json	625	86a30e75f291e351
frontend/public/favicon.svg	479	6fbded83639bc1db
frontend/src/main.jsx	34470	89ef27389c6aaf9a
frontend/src/styles.css	21986	533feafd3fef6f2a
goldens.json	11315	624153833c4a73aa
graph.png	34976	6bbaf4791fcae607
main.py	89	9b71be760015e244
pyproject.toml	863	092b346978b0dc24
qdrant_login_steps.md	1890	98d6444b252619a3
requirements.txt	3631	19cc834b44d92a16
sessions.json	211	049a1016b4df9d83
smoke_features.py	12320	41e6caf69efaee71
uv.lock	795461	51b955192cc42322

H. Executed offline checks

AST-isolated source functions with deterministic stubs; not full integration tests.

- PASS: All 28 first-party Python files parse.
- PASS: Empty reranker model name falls back to base model.
- PASS: Bare reranker name receives BAAI prefix.
- PASS: Empty candidate list bypasses model.
- PASS: Reranker truncates scoring text to configured characters.
- PASS: Descending score and top-N selection.
- PASS: Candidate and rerank positions retained.
- PASS: Original candidate metadata not mutated.
- PASS: arXiv loader strips requested version.
- PASS: Legacy arXiv identifier is not parsed.
- PASS: SSE event framing and JSON payload.

Passed: npm run build -- --outDir /tmp/scholarlens-documentation-build; 1859 modules; large JS chunk warning.

Not executed: backend API startup, real model inference, Qdrant access, Tavily search, live smoke_features.py, evaluate.py, Docker startup. The PDF itself is compiled, text-extracted, and visually checked separately.

H. Recommended acceptance tests for a future validated deployment

- Startup with each explicitly supported provider and a known embedding model, including missing-key/model failures.
- Known-document upload, repeated upload, unsupported format, empty/scanned content, URL failure, and arXiv version behavior.
- Session isolation plus actual ownership enforcement if added; concurrent create/rename/delete handling.
- Collection schema mismatch with a populated test collection, ensuring a migration does not erase evidence unexpectedly.
- Deterministic rerank ordering and metadata, then real ranking evaluation against a held-out set.
- SSE token/replace/final/error handling, Unicode splits, interrupted EOF, abort, retry, and session switches during streaming.
- Per-turn evidence persistence, internal-query rewrite visibility, and complete deletion/retention behavior.
- Docker fresh-volume provisioning, model cache persistence, GPU/CPU operation, and backup/restore of all state stores.

These are proposed tests, not claimed passes.